CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 14: Single-Cycle Datapath I

Instructor: Ariana Abel

Slides Credit: Justin Yokota

Ariana Abel

- CS & Math Major
- Super Senior 😆
- 5th time teaching 61C
- I <3 computer hardware
- Interests: **C**rocheting, **C**offee, and **C**oncerts

Agenda

- Building a RISC-V Processor
- CPU Elements and Stages
- R-Type: `add` Datapath
- R-Type: `sub` Datapath
- Datapath with immediates: `addi`
- Implementing Loads
- Implementing Stores
- Implementing Jumps

Agenda

- **Building a RISC-V Processor**
- CPU Elements and Stages
- R-Type: `add` Datapath
- R-Type: `sub` Datapath
- Datapath with immediates: `addi`
- Implementing Loads
- Implementing Stores
- Implementing Jumps

Great Idea #1: Abstraction (Levels of Representation/Interpretation)

High Level Language Program (e.g., C)

Compiler

Assembly Language Program (e.g., RISC-V)

Assembler

Machine Language Program (RISC-V)

Hardware Architecture Description (e.g., block diagrams)

Architecture Implementation

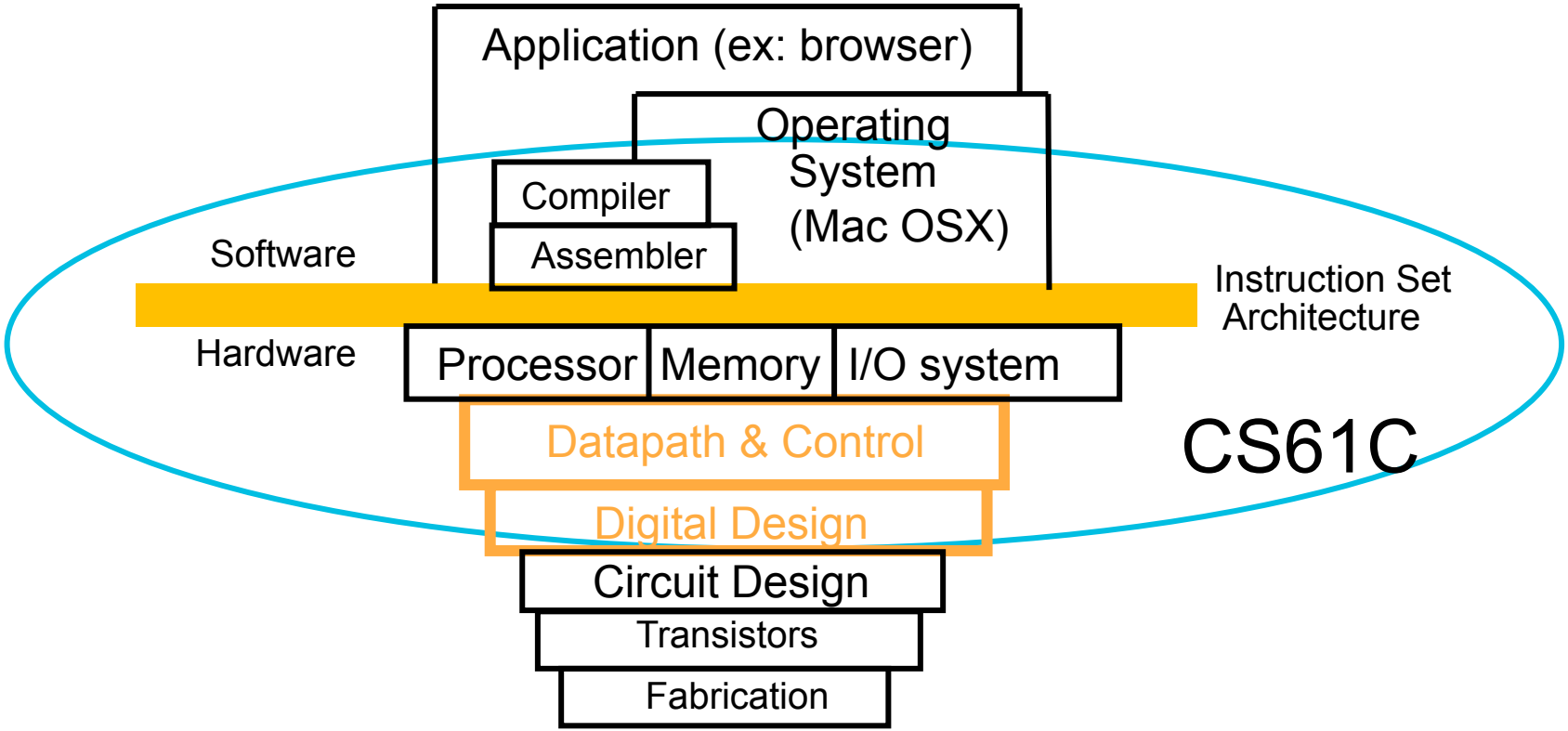
Logic Circuit Description (Circuit Schematic Diagrams)

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw    x3, 0(x10)
lw    x4, 4(x10)
sw    x4, 0(x10)
sw    x3, 4(x10)
```

```
1000 1101 1110 0010 0000 0000 0000 0000
1000 1110 0001 0000 0000 0000 0000 0100
1010 1110 0001 0010 0000 0000 0000 0000
1010 1101 1110 0010 0000 0000 0000 0100
```

Old-school Machine Structures



New-school Machine Structures

Software

Harness Parallelism & Achieve High Performance

Parallel Requests
Assigned to computer
e.g., Search “Cats”

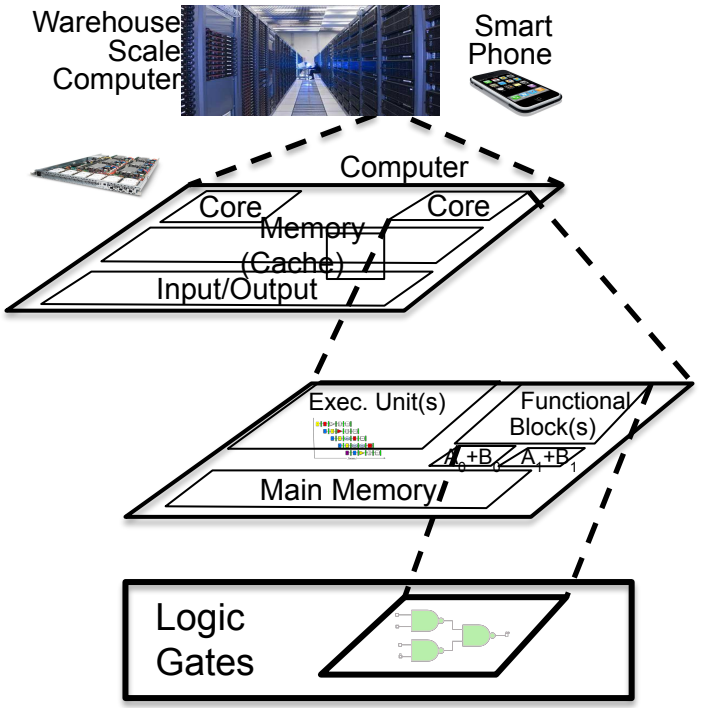
Parallel Threads
Assigned to core e.g., Lookup, Ads

Parallel Instructions
>1 instruction @ one time
e.g., 5 pipelined instructions

Parallel Data
>1 data item @ one time
e.g., Add of 4 pairs of words

Hardware descriptions
All gates work in parallel at same time

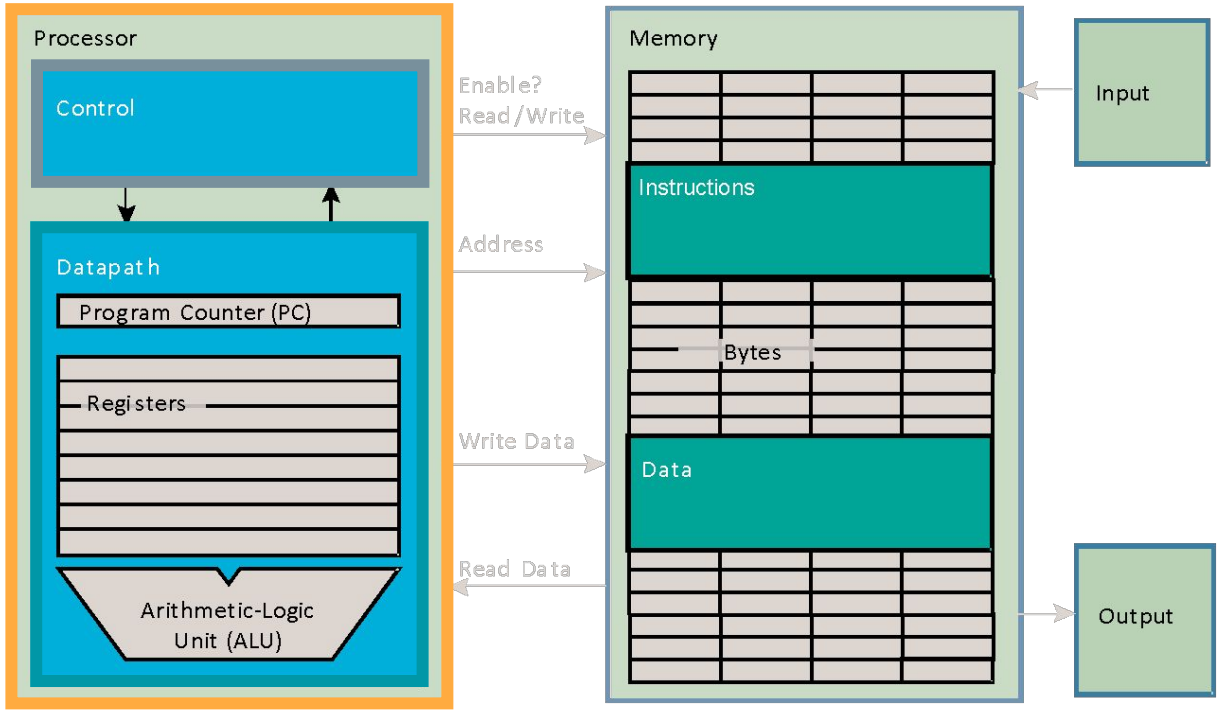
Hardware



How do we build a Single-Core Processor?

Processor (CPU): the active part of the computer that does all the work (data manipulation and decision-making).

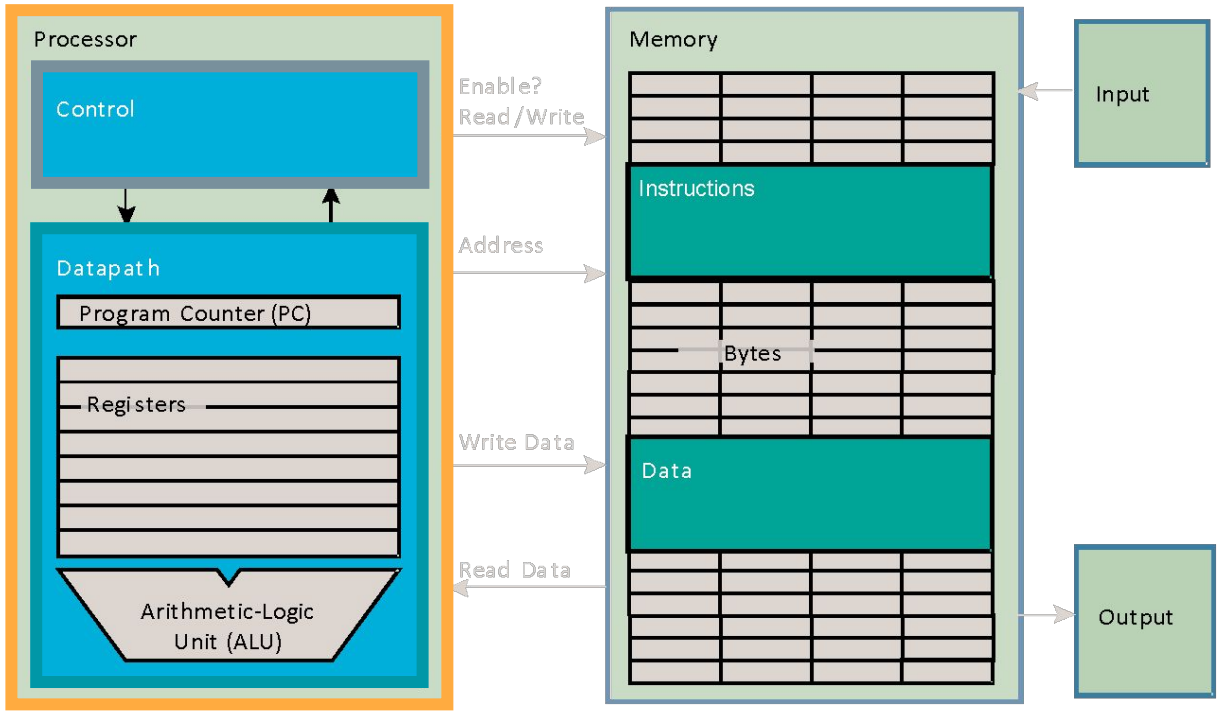
- Ultimately just a complicated circuit



How do we build a Single-Core Processor?

Datapath ("the brawn"): portion of the processor that contains hardware necessary to perform operations required by the processor.

Control ("the brain"): portion of the processor (also in hardware) that tells the datapath what needs to be done.



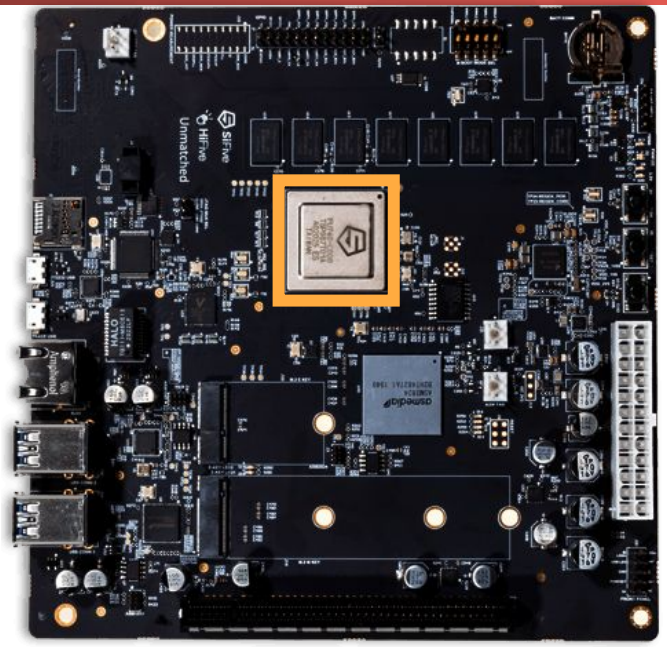
Goal: Design HW to Execute All RV32I Instructions

Open RISC-V Reference Card

Base Integer Instructions: RV32I

Category	Name	Fmt	RV32I Base	Category	Name	Fmt	RV32I Base
Shifts	Shift Left Logical	R	SLL rd,rs1,rs2	Loads	Load Byte	I	LB rd,rs1,imm
	Shift Left Log. Imm.	I	SLLI rd,rs1,shamt		Load Halfword	I	LH rd,rs1,imm
	Shift Right Logical	R	SRL rd,rs1,rs2		Load Byte Unsigned	I	LBU rd,rs1,imm
	Shift Right Log. Imm.	I	SRLI rd,rs1,shamt		Load Half Unsigned	I	LHU rd,rs1,imm
	Shift Right Arithmetic	R	SRA rd,rs1,rs2		Load Word	I	LW rd,rs1,imm
	Shift Right Arith. Imm.	I	SRAI rd,rs1,shamt	Stores	Store Byte	S	SB rs1,rs2,imm
Arithmetic	ADD	R	ADD rd,rs1,rs2		Store Halfword	S	SH rs1,rs2,imm
	ADD Immediate	I	ADDI rd,rs1,imm		Store Word	S	SW rs1,rs2,imm
	SUBtract	R	SUB rd,rs1,rs2	Branches	Branch =	B	BEQ rs1,rs2,imm
	Load Upper Imm	U	LUI rd,imm		Branch ≠	B	BNE rs1,rs2,imm
	Add Upper Imm to PC	U	AUIPC rd,imm		Branch <	B	BLT rs1,rs2,imm
Logical	XOR	R	XOR rd,rs1,rs2		Branch ≥	B	BGE rs1,rs2,imm
	XOR Immediate	I	XORI rd,rs1,imm		Branch < Unsigned	B	BLTU rs1,rs2,imm
	OR	R	OR rd,rs1,rs2		Branch ≥ Unsigned	B	BGEU rs1,rs2,imm
	OR Immediate	I	ORI rd,rs1,imm	Jump & Link	J&L	J	JAL rd,imm
	AND	R	AND rd,rs1,rs2		Jump & Link Register	I	JALR rd,rs1,imm
	AND Immediate	I	ANDI rd,rs1,imm				
Compare	Set <	R	SLT rd,rs1,rs2	Synch	Synch thread	I	FENCE
	Set < Immediate	I	SLTI rd,rs1,imm				
	Set < Unsigned	R	SLTU rd,rs1,rs2	Environment	CALL	I	ECALL
	Set < Imm Unsigned	I	SLTIU rd,rs1,imm		BREAK	I	EBREAK

Not in 61C



HiFive Unmatched (RISC-V Linux)

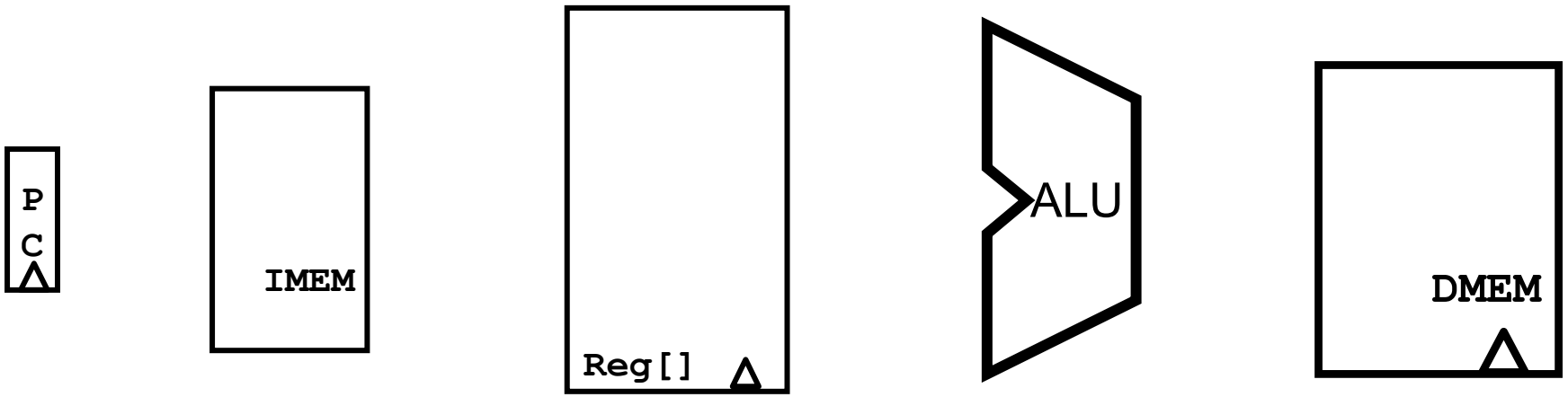
https://www.sifive.com/boards/hifive-unmatched

Agenda

- Building a RISC-V Processor
- **CPU Elements and Stages**
- R-Type: `add` Datapath
- R-Type: `sub` Datapath
- Datapath with immediates: `addi`
- Implementing Loads
- Implementing Stores
- Implementing Jumps

One-Instruction-Per-Cycle RISC V Machine

The CPU is composed of two types of subcircuits: **combinational logic blocks** and **state elements**.



One-Instruction-Per-Cycle RISC V Machine

- On **every tick of the clock**, the computer executes one instruction:
  - Current outputs of the state elements drive the inputs to combinational logic
- At the **rising clock edge**:
  - All the state elements are updated with the combinational logic outputs…
  - and execution moves to the next clock cycle.
- The **state elements** determine the behavior of the CPU

Combinational Logic Blocks

- Any function can be built using **AND, OR, and NOT** gates.
- Examples:
  - Adder
  - Multiplexer (MUX): selects one input based on a selector.
- Wires carry 0 or 1 (low vs high voltage).
- All logic runs simultaneously (not controlled by clock)
- Each logic block has a **delay**
  - how long it takes for that block to compute its function

State Elements

- State elements store the current *data* of the program
- RISC-V is designed to work according to state elements ONLY
  - E.g. If two CPUs have the exact same values in all state elements, then both CPUs will perform the exact same sequence of operations. What the CPU does is determined *solely* by a current snapshot of all state elements.
- Every instruction modifies some state elements (otherwise the program would infinite-loop)
- Three main state elements in the CPU:
  - Registers (PC)
  - RegFile
  - Main Memory

State Element: Register

Input:

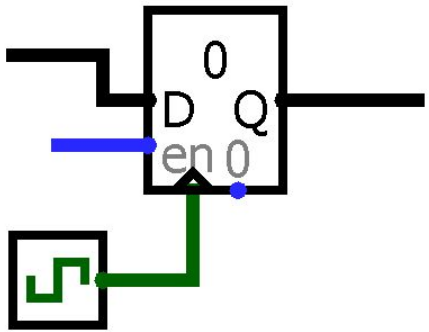
- 32-bit data input bus (D)
- Write Enable “Control” bit (1 = Enabled, 0 = Disabled)

Output:

- 32-bit data output bus (Q)

Behavior:

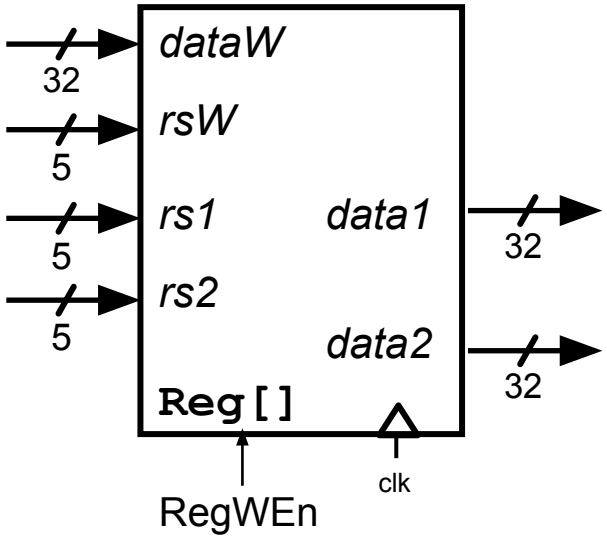
- If Write Enable is 1 on rising clock edge, set Data Out=Data In.
- At **all other times**, Data Out will not change; it will output its current value.
- Note: **Reading** the value of a register doesn't take a clock tick. Only **Writing** to the register takes a tick.



State Element: Register File

Group of 32 registers; manages which registers get changed and which register outputs get read

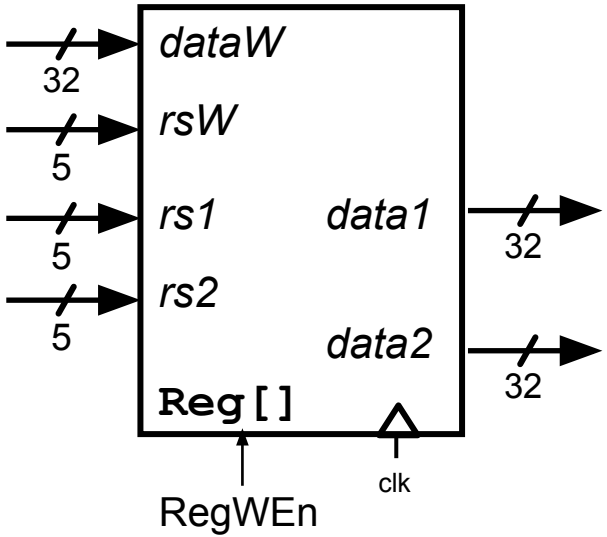
- Input:
  - One 32-bit input bus, *dataW*.
  - Three 5-bit selectors, *rs1*, *rs2*, and *rsW*.
  - RegWEn control bit (Write Enabled).
- Output:
  - Two 32-bit output busses, *data1* and *data2* (The data in rs1 and rs2).



State Element: Register File

Registers are accessed via their 5-bit register numbers:

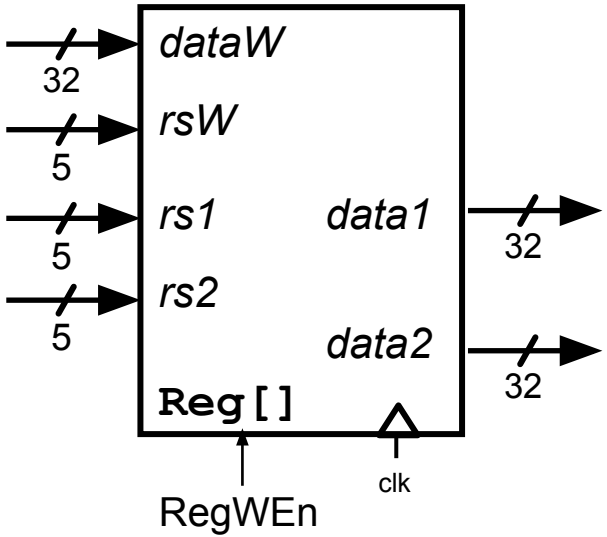
- rs1: 5-bit number that identifies which register should output in data1
- rs2: 5-bit number that identifies which register should output in data2
- rsW: 5-bit number that identifies which register should update next cycle



State Element: Register File

Clock behavior:

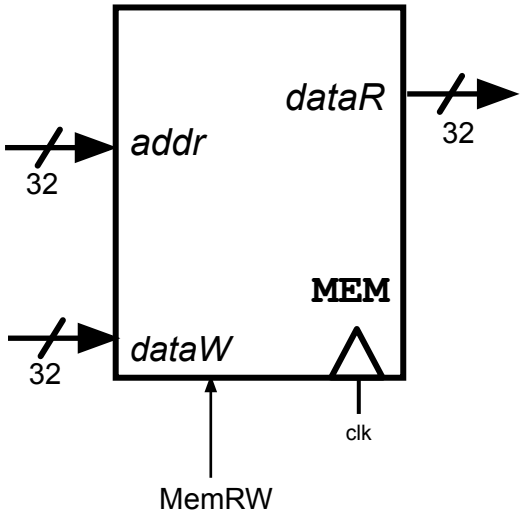
- If **RegWEn** is 1 on rising clock edge, set rsW=DataW.
- **Reading** the value of a register doesn't take a clock tick. Only **Writing** to the register takes a tick.



State Element: Memory

Main memory: Allows access to each byte saved in the program

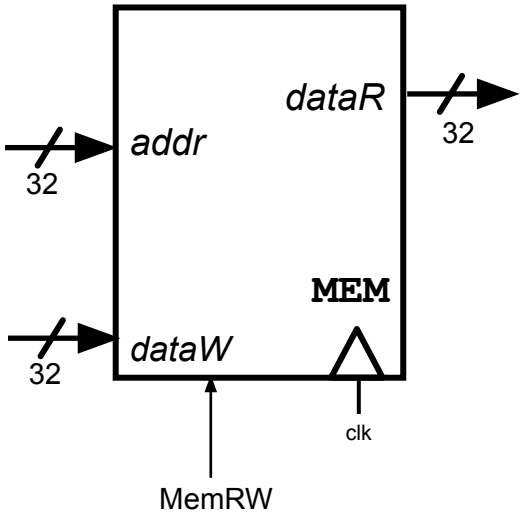
- Input:
  - One 32-bit input bus, *addr*.
  - One 32-bit input bus, *dataW*
  - MemRW control bit (Write Enabled).
- Output:
  - One 32-bit output bus, *dataR* (The word stored at *addr*).



State Element: Memory

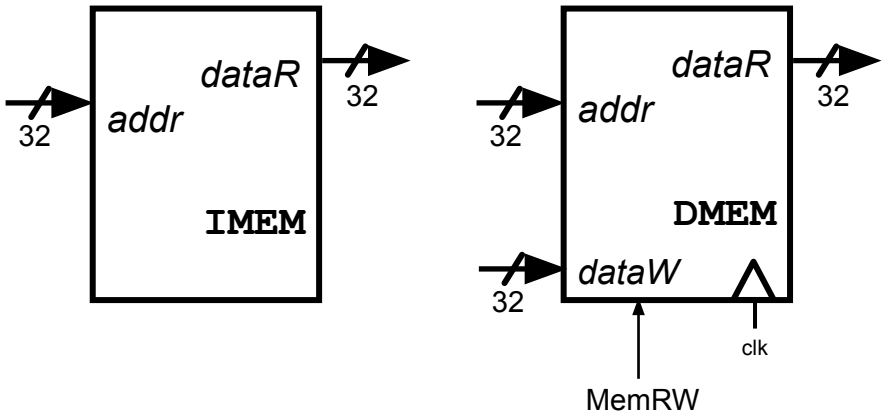
For now, main memory is "magic".

- Acts *similar* to the Regfile (ex. put in an address, get out the data at that address).
  - Clock behaves the same way
- But registers are too expensive, so some other underlying technology gets used.



State Element: IMEM vs DMEM

- Current abstraction: Memory holds both instructions and data in one contiguous 32-bit memory space.
- In our processor, we'll use two "separate" memories:
  - **`IMEM`**: A *read-only* memory for fetching instructions.
  - **`DMEM`**: A memory for loading (read) and storing (write) data words.
  - Under the hood, these are placeholders for caches.
- Because IMEM is read-only, it never requires a clock trigger, dataW, etc.

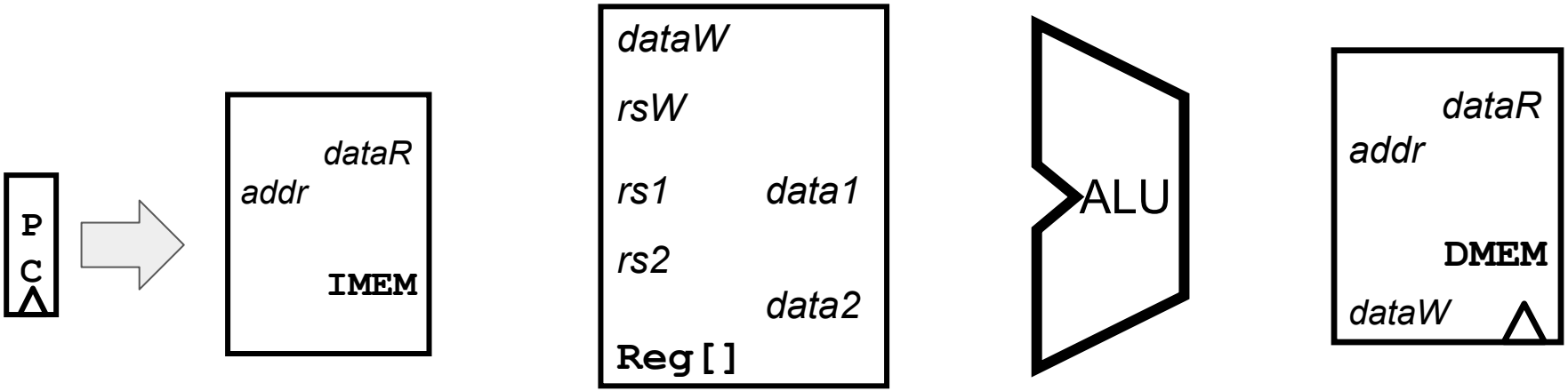


Designing the Datapath

All instructions follow the same general 5 "stages" of computation

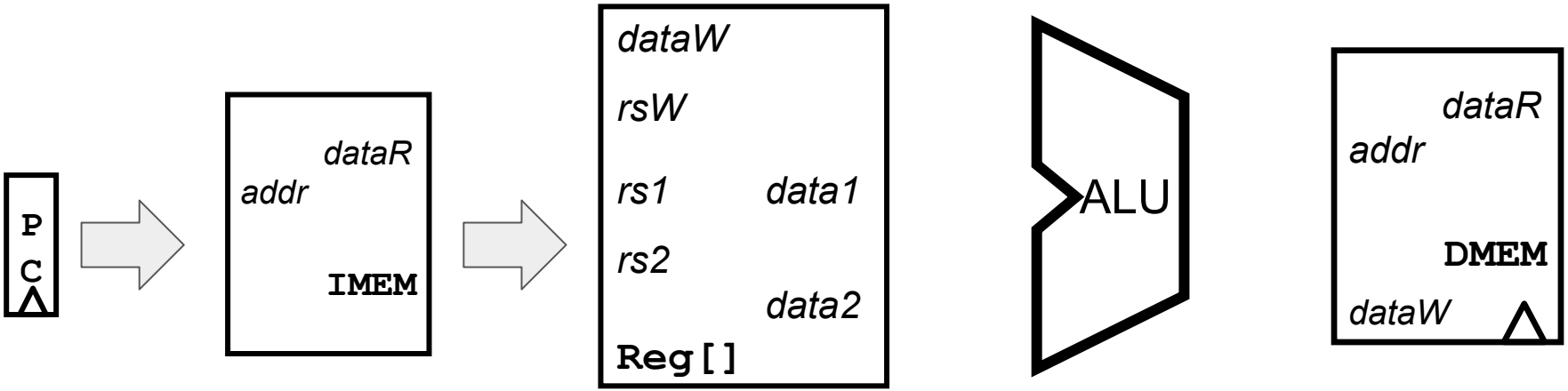
Designing the Datapath

"Instruction Fetch" (IF): Send PC into IMEM, retrieve the instruction to be run.



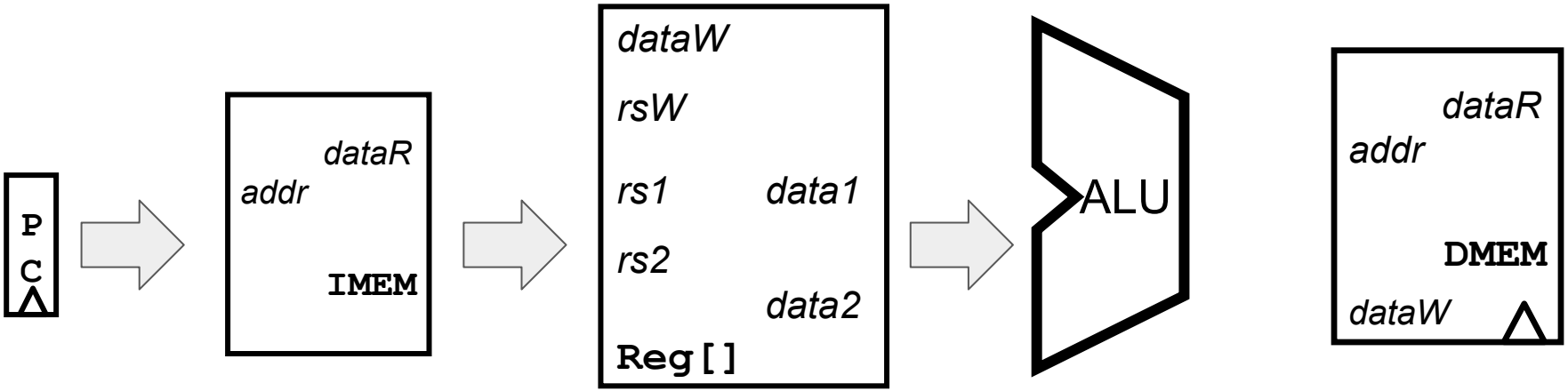
Designing the Datapath

"Instruction Decode" (ID): Read instruction, decode what the instruction is meant to do, compute immediates/register values.



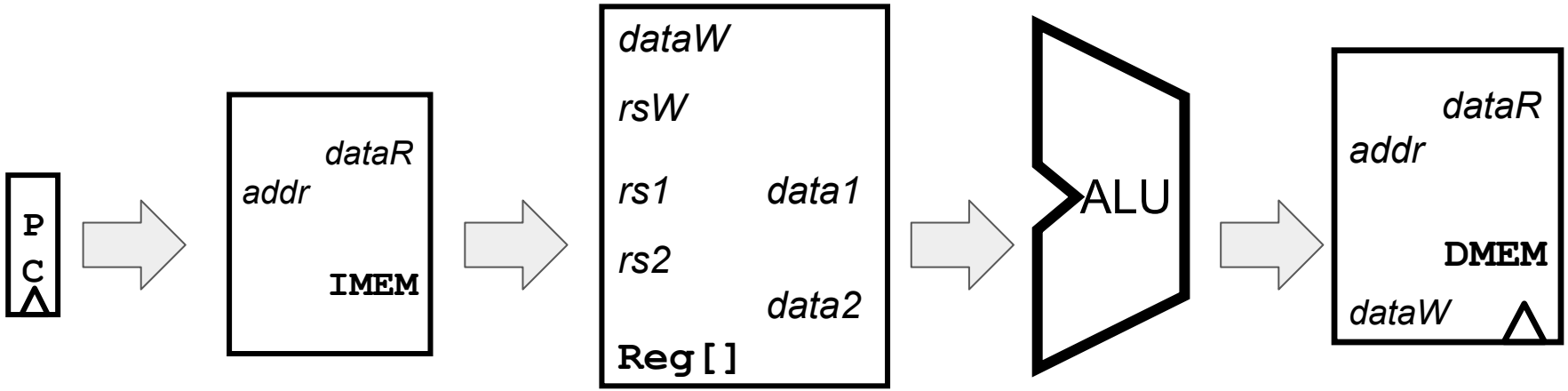
Designing the Datapath

"Execute" (EX): Do any computations needed for the instruction.



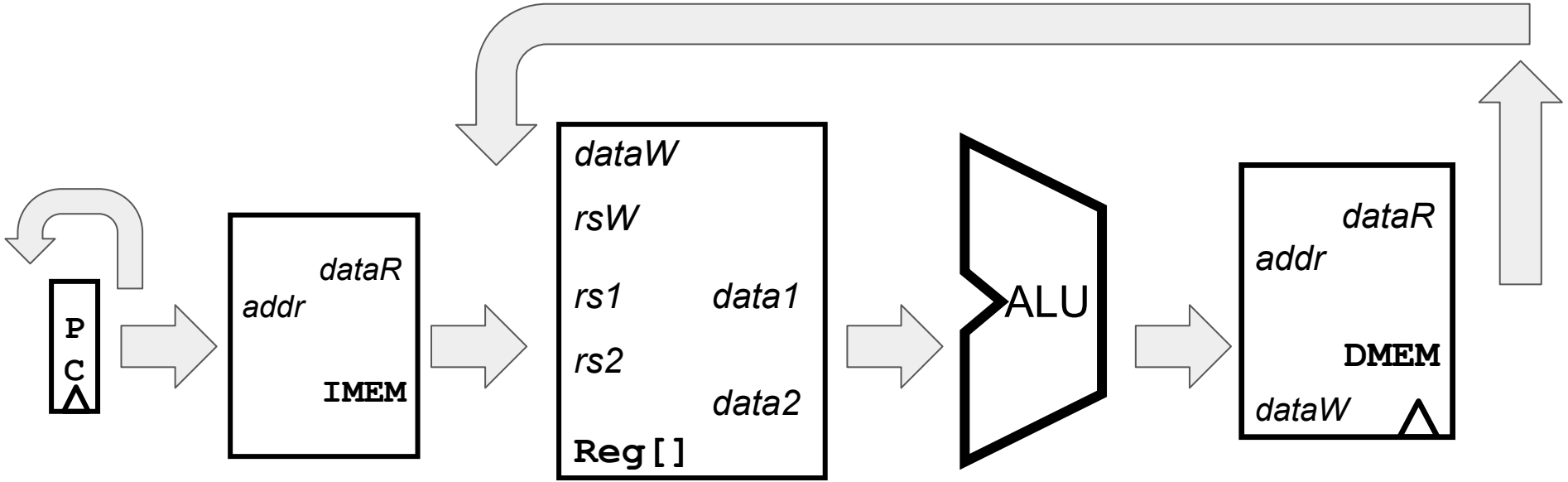
Designing the Datapath

"Memory" (MEM): For DMEM operations, a separate stage for accessing memory.



Designing the Datapath

"Write Back" (WB): Update inputs to any state elements that need to be updated, wait for next rising edge to actually write data back



Designing the Datapath

All instructions follow the same general "stages" of computation:

- "Instruction Fetch" (IF): Send PC into IMEM, retrieve the instruction to be run.
- "Instruction Decode" (ID): Read instruction, decode what the instruction is meant to do, compute immediates/register values.
- "Execute" (EX): Do any computations needed for the instruction.
  - "Memory" (MEM): For DMEM operations, a separate stage for accessing memory.
- "Write Back" (WB): Update inputs to any state elements that need to be updated, wait for next rising edge to actually write data back

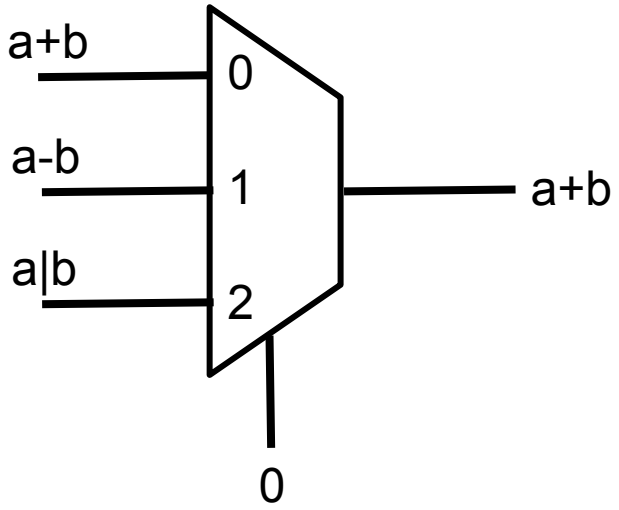
For a single-stage datapath, all stages happen in **a single clock cycle**.

Datapath Design Philosophy

- Every logic block/state element in a CPU needs to do something each cycle, but all blocks run simultaneously.
- General philosophy: do **every single operation** for every instruction, even if the result doesn't make sense. But add **control logic** to ensure that only the operations we *want* end up affecting state elements.

Datapath Design Philosophy

- Important tool here is the **multiplexer (mux)**: Receive multiple inputs (ex. a+b, a-b, a|b) and a selector (ex. "I want the ADD result"), and output the corresponding input (a+b).



Datapath Design Philosophy

- For the rest of this lecture and Thursday's: Building the datapath, one instruction at a time.

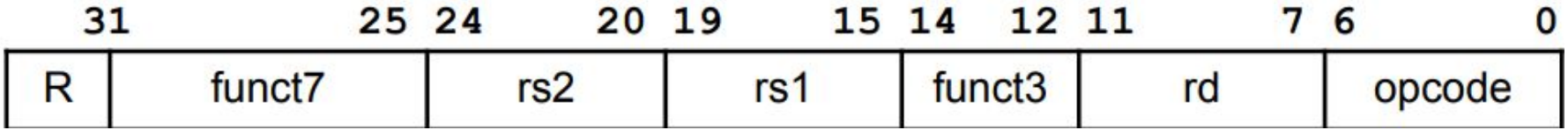
Agenda

- Building a RISC-V Processor
- CPU Elements and Stages
- **R-Type: `add` Datapath**
- R-Type: `sub` Datapath
- Datapath with immediates: `addi`
- Implementing Loads
- Implementing Stores
- Implementing Jumps

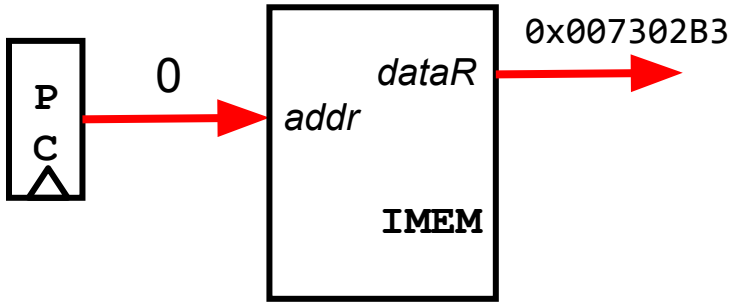
Implementing the add instruction

- For now, let's imagine that the only instruction we have is "add"
- Since add doesn't use memory, we will not use a DMEM yet. But all other parts of the datapath are needed.
- add does two things to state elements:
  - Sets rd to rs1+rs2
  - Sets PC to PC+4

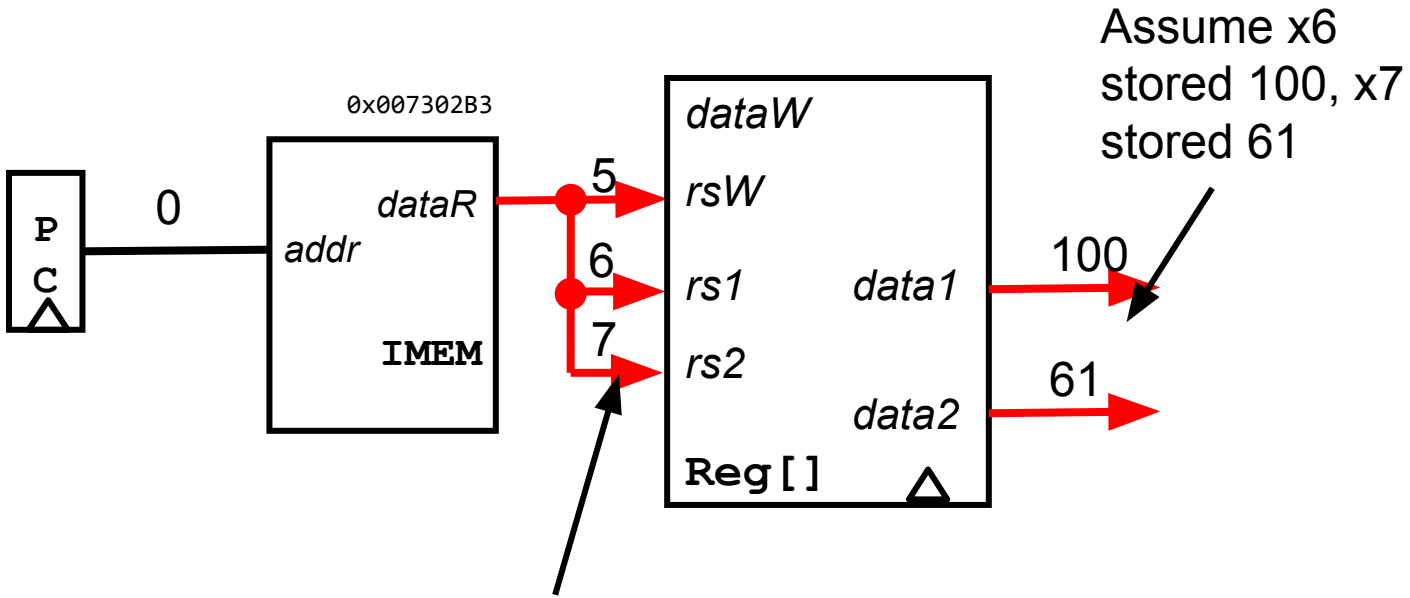
Instruction	Name	Description	Type	Opcode	Funct3	Funct7
`add rd rs1 rs2`	ADD	`rd = rs1 + rs2`	R	`011 0011`	`000`	`000 0000`



Implementing add (example: add x5 x6 x7): IF

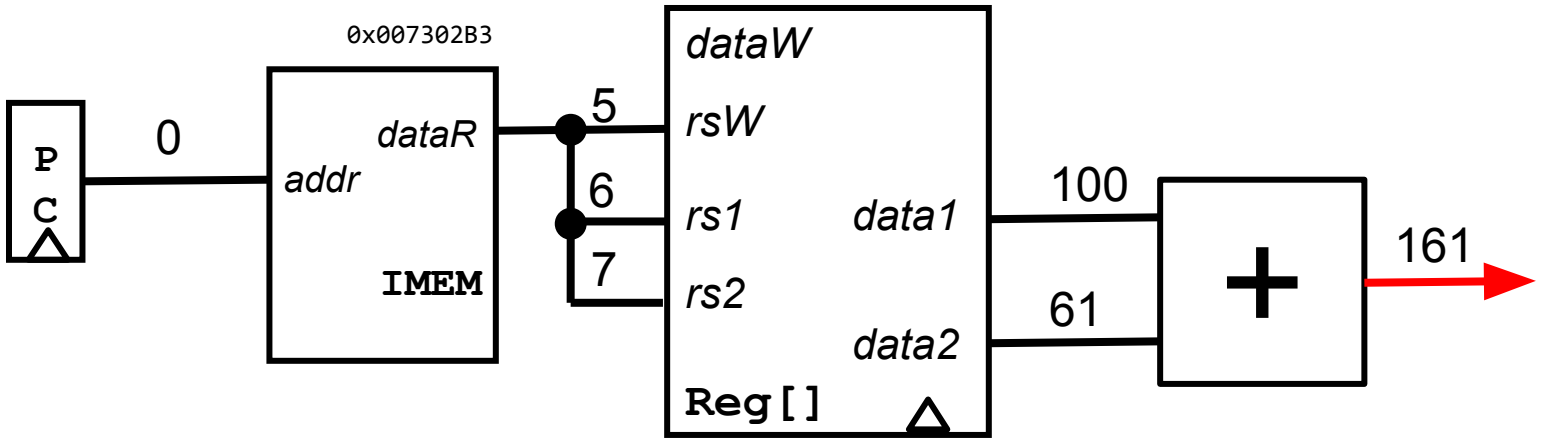


Implementing add (example: add x5 x6 x7): ID

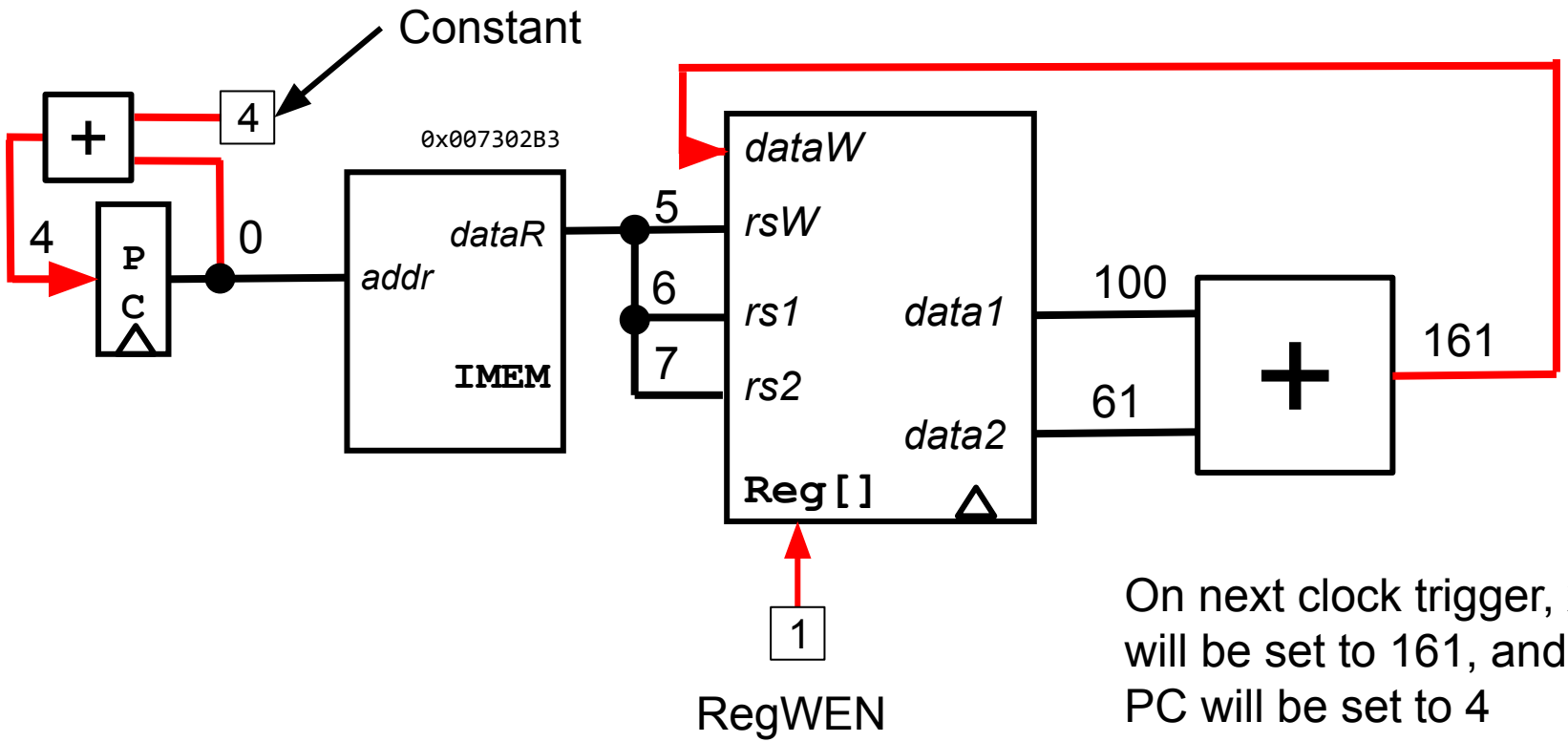


Note: rs1 always inst[19:15], rs2 always inst[24:20], rd always inst[11:7] if they exist

Implementing add (example: add x5 x6 x7): EX



Implementing add (example: `add x5 x6 x7`): WB



On next clock trigger, x5 will be set to 161, and PC will be set to 4

Agenda

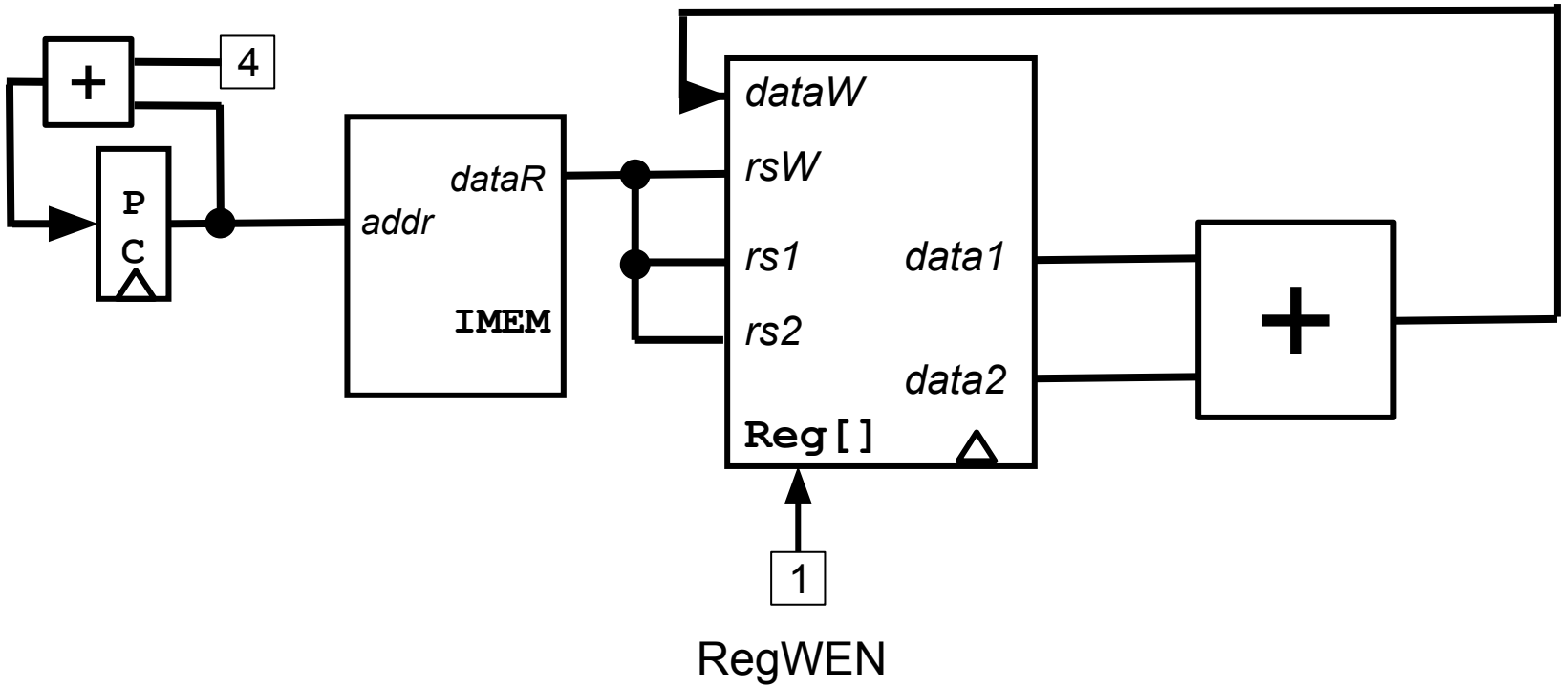
- Building a RISC-V Processor
- CPU Elements and Stages
- R-Type: `add` Datapath
- **R-Type: `sub` Datapath**
- Datapath with immediates: `addi`
- Implementing Loads
- Implementing Stores
- Implementing Jumps

Implementing the `sub` instruction

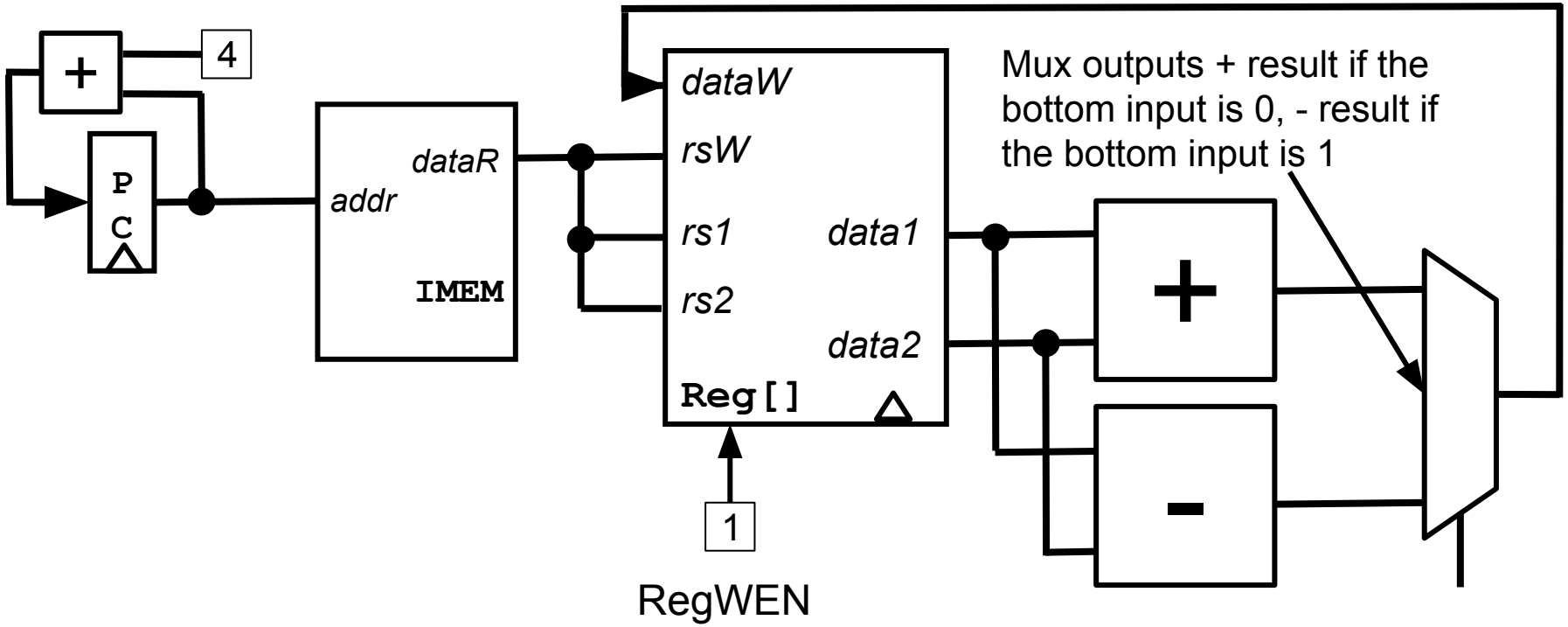
- Let's add another R-type instruction: `sub`
- `sub` does two things to state elements:
  - Sets rd to rs1-rs2
  - Sets PC to PC+4
- Main idea: Do both addition and subtraction, and use a mux to decide which one to output

Instruction	Name	Description	Type	Opcode	Funct3	Funct7
`add rd rs1 rs2`	ADD	`rd = rs1 + rs2`	R	`011 0011`	`000`	`000 0000`
`sub rd rs1 rs2`	SUBtract	`rd = rs1 - rs2`	R	`011 0011`	`000`	`010 0000`

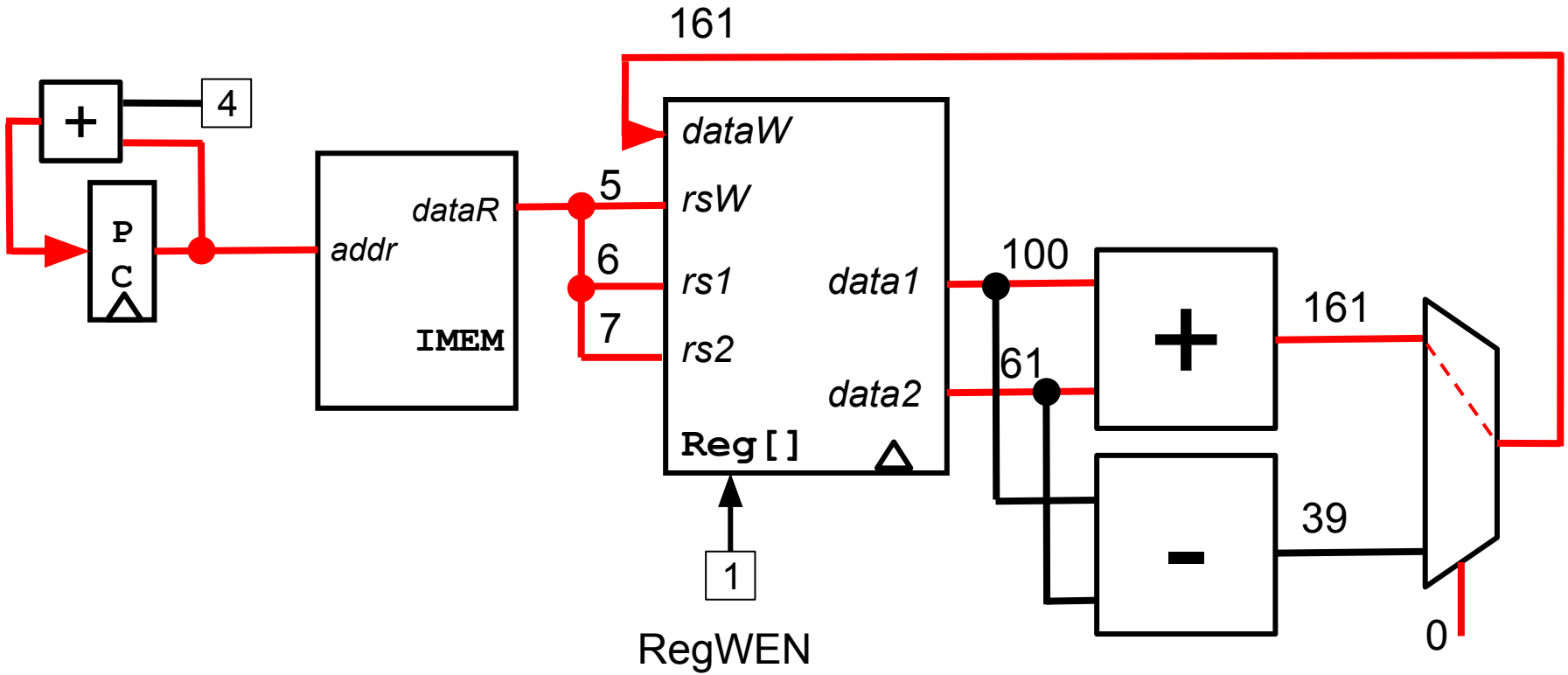
add Datapath: What do you need to add to handle sub?



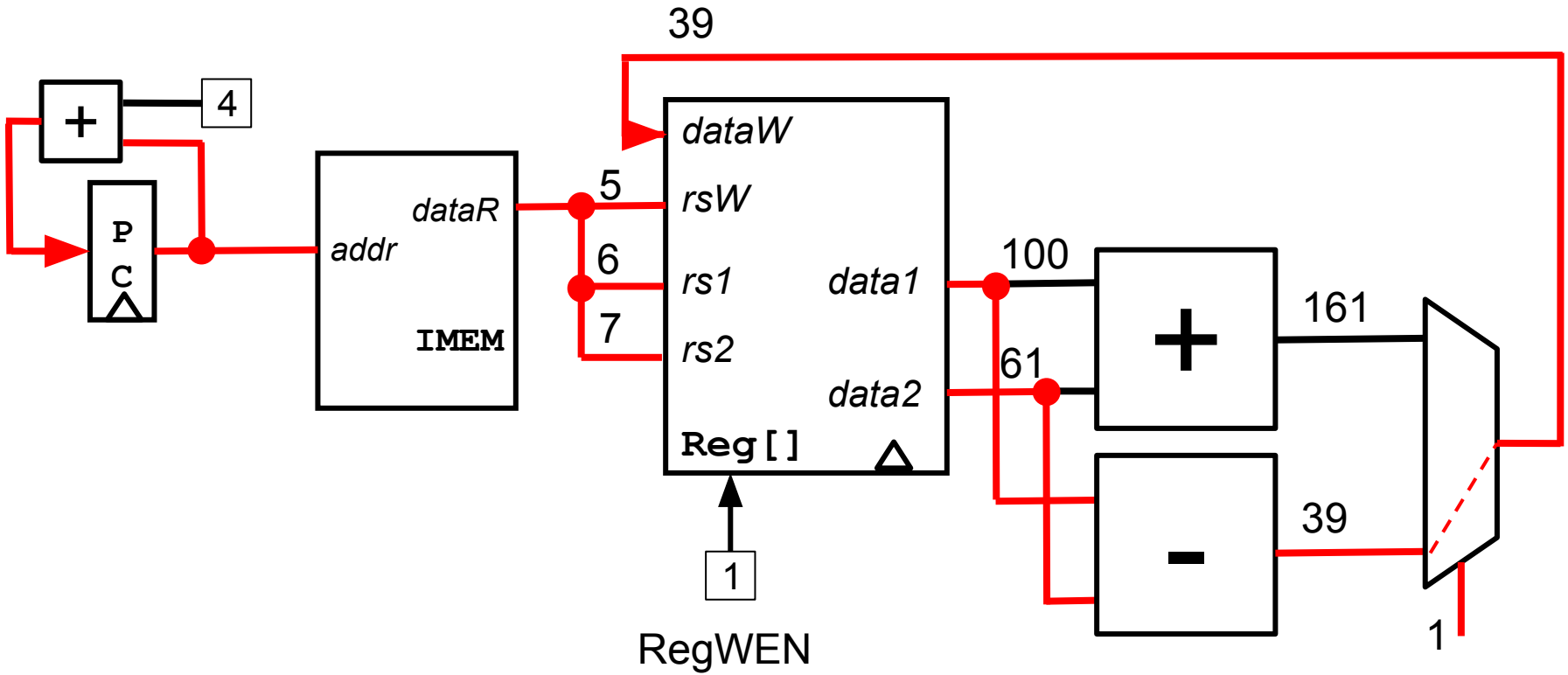
add/sub Datapath



add/sub Datapath running add x5 x6 x7



add/sub Datapath running sub x5 x6 x7

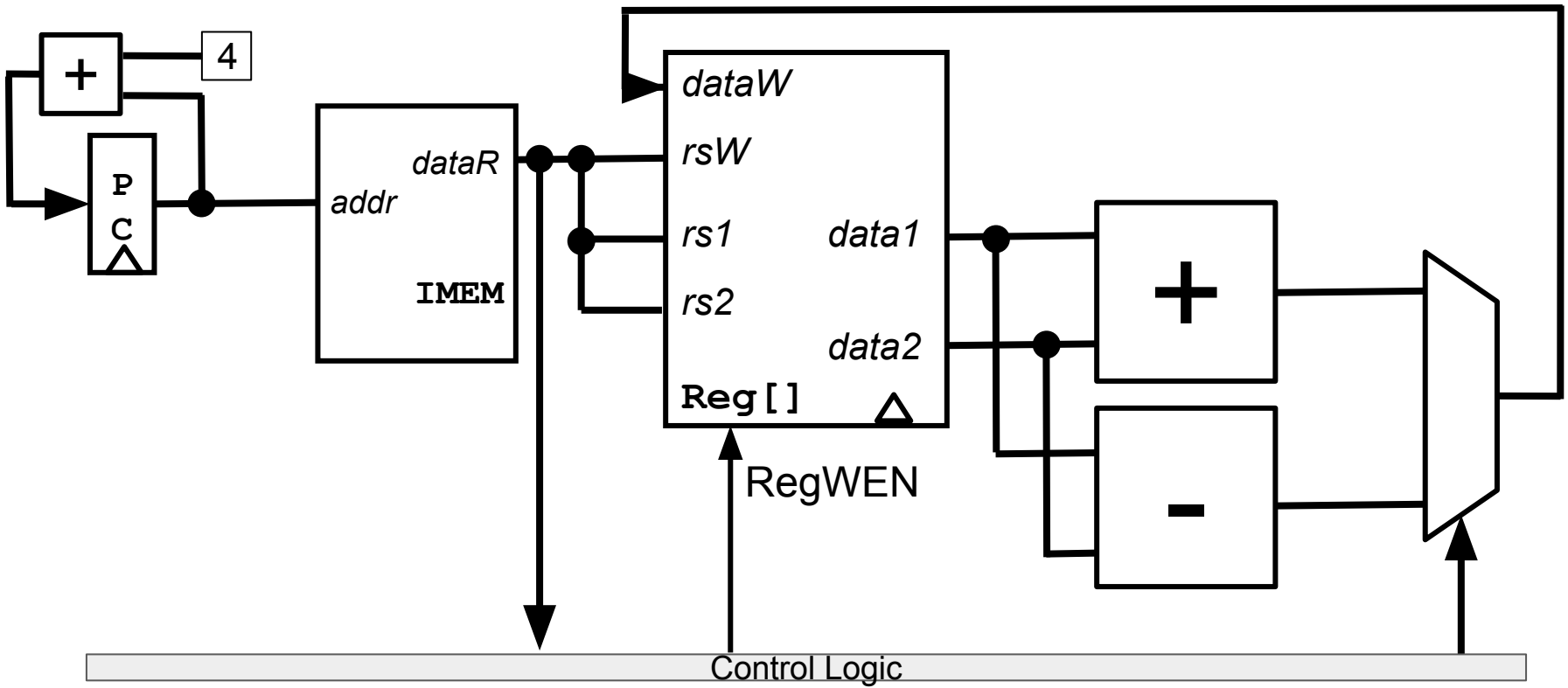


Deciding mux selectors

- How do we decide what selector to set for the mux?
  - Use the differences in opcode, funct3, or funct7
- In this case: Bit 6 of the funct7 is 1 for `sub`, 0 for `add`. So we can send bit 30 of the instruction (which is bit 6 of funct7) to the mux selector
- For more complicated cases: We'll abstract it away for now in a combinational logic block we'll call the **control logic**
  - Control logic takes in the instruction, and outputs all mux selectors and WENs needed for the instruction

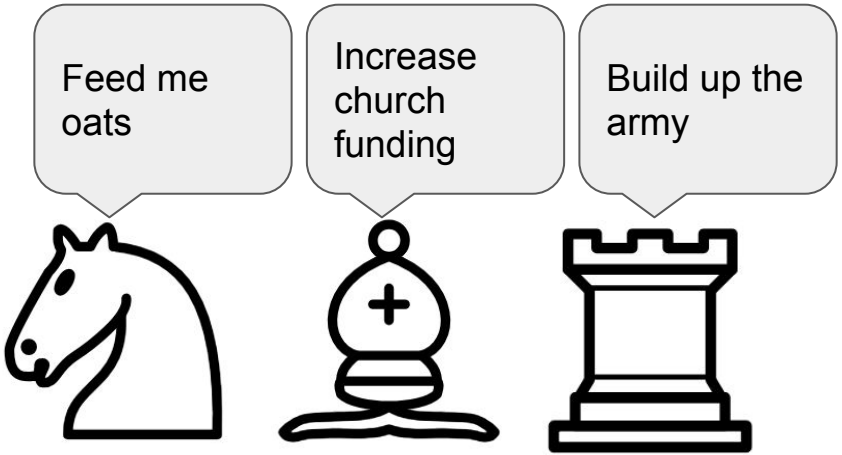
Instruction	Name	Description	Type	Opcode	Funct3	Funct7
`add rd rs1 rs2`	ADD	`rd = rs1 + rs2`	R	`011 0011`	`000`	`000 0000`
`sub rd rs1 rs2`	SUBtract	`rd = rs1 - rs2`	R	`011 0011`	`000`	`010 0000`

add/sub Datapath



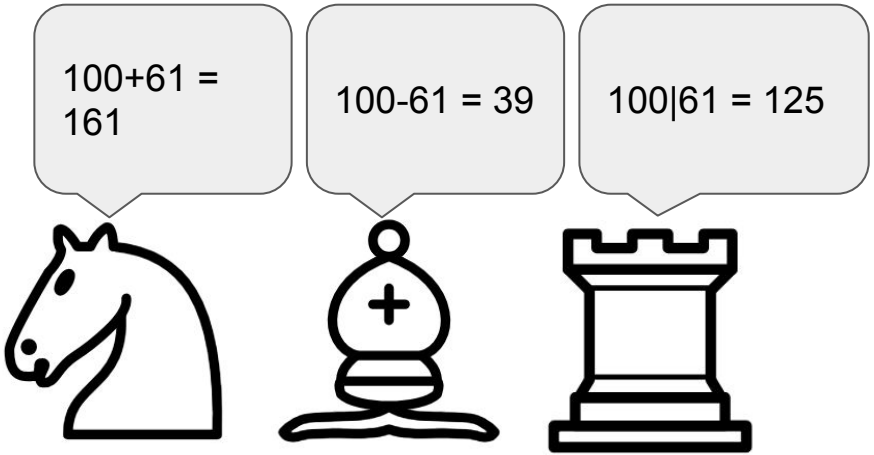
Analogy: A king and his advisors

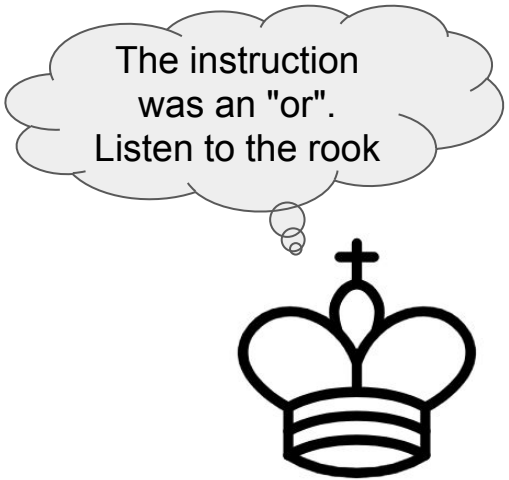
- The datapath is a group of advisors: they do math according to their specialization and report their results
- The control logic is the king: they decide which advisor to listen to, based on what the kingdom needs.



Analogy: A king and his advisors

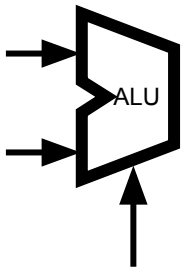
- The datapath is a group of advisors: they do math according to their specialization and report their results
- The control logic is the king: they decide which advisor to listen to, based on what the kingdom needs.





Supporting all R-Type Instructions

- Create a subcircuit called "ALU" (Arithmetic Logic Unit) to handle all math. Control logic decides which operation the ALU does.



ALUSel

Instruction	Name	Description	Type	Opcode	Funct3	Funct7
`add rd rs1 rs2`	ADD	`rd = rs1 + rs2`	R	`011 0011`	`000`	`000 0000`
`sub rd rs1 rs2`	SUBtract	`rd = rs1 - rs2`	R	`011 0011`	`000`	`010 0000`
`and rd rs1 rs2`	bitwise AND	`rd = rs1 & rs2`	R	`011 0011`	`111`	`000 0000`
`or rd rs1 rs2`	bitwise OR	`rd = rs1 | rs2`	R	`011 0011`	`110`	`000 0000`
`xor rd rs1 rs2`	bitwise XOR	`rd = rs1 ^ rs2`	R	`011 0011`	`100`	`000 0000`
`sll rd rs1 rs2`	Shift Left Logical	`rd = rs1 << rs2`	R	`011 0011`	`001`	`000 0000`
`srl rd rs1 rs2`	Shift Right Logical	`rd = rs1 >> rs2` (Zero-extend)	R	`011 0011`	`101`	`000 0000`
`sra rd rs1 rs2`	Shift Right Arithmetic	`rd = rs1 >> rs2` (Sign-extend)	R	`011 0011`	`101`	`010 0000`
`slt rd rs1 rs2`	Set Less Than (signed)	`rd = (rs1 < rs2) ? 1 : 0`	R	`011 0011`	`010`	`000 0000`
`sltu rd rs1 rs2`	Set Less Than (Unsigned)		R	`011 0011`	`011`	`000 0000`

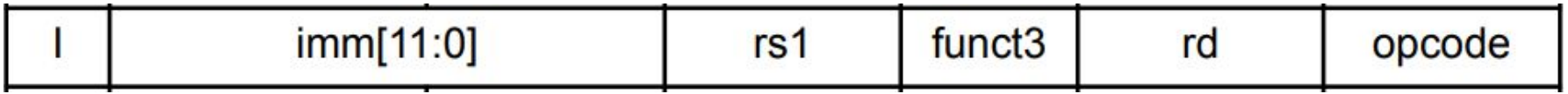
Agenda

- Building a RISC-V Processor
- CPU Elements and Stages
- R-Type: `add` Datapath
- R-Type: `sub` Datapath
- **Datapath with immediates: `addi`**
- Implementing Loads
- Implementing Stores
- Implementing Jumps

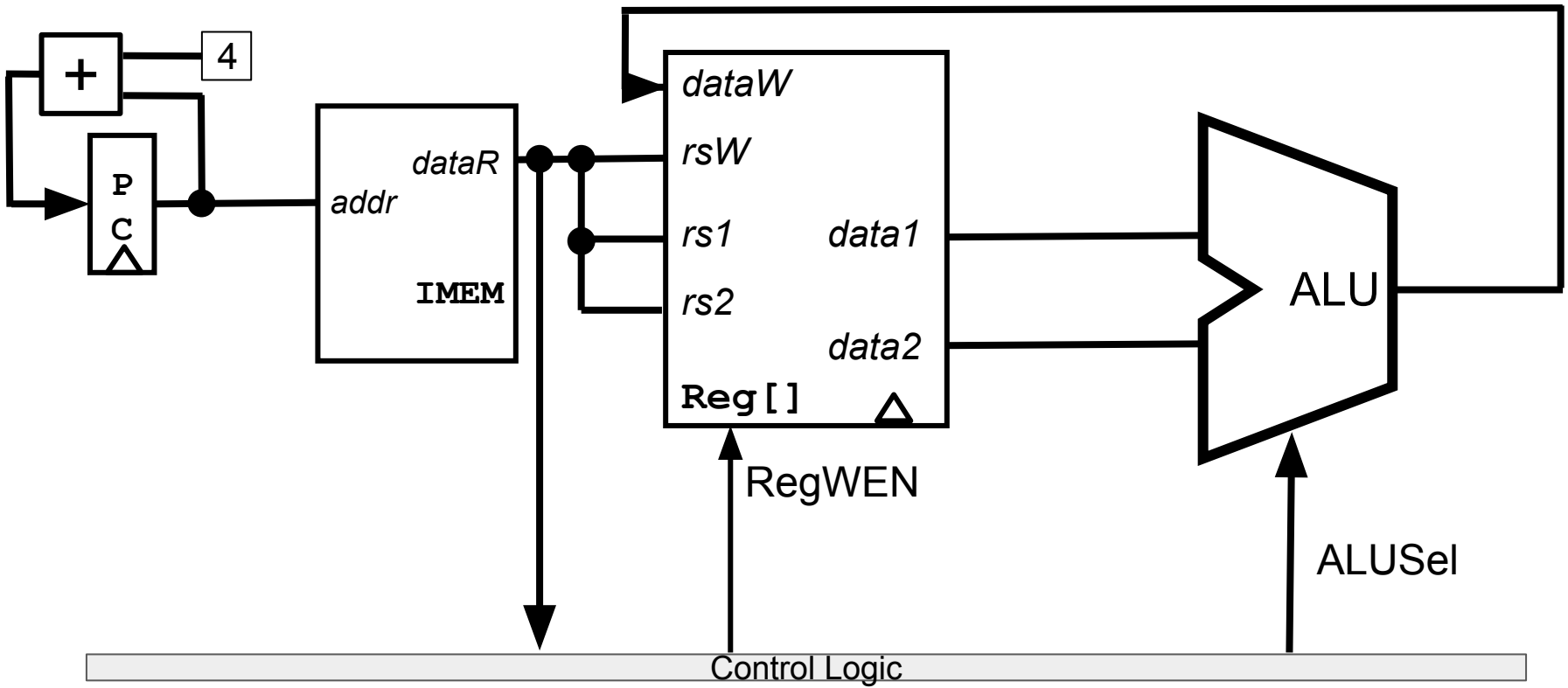
Implementing the `addi` instruction

- Let's add an I-type instruction: `addi`
- `addi` does two things to state elements:
  - Sets rd to rs1+imm
  - Sets PC to PC+4
- Same as `add`, but now we need a subcircuit to construct the immediate

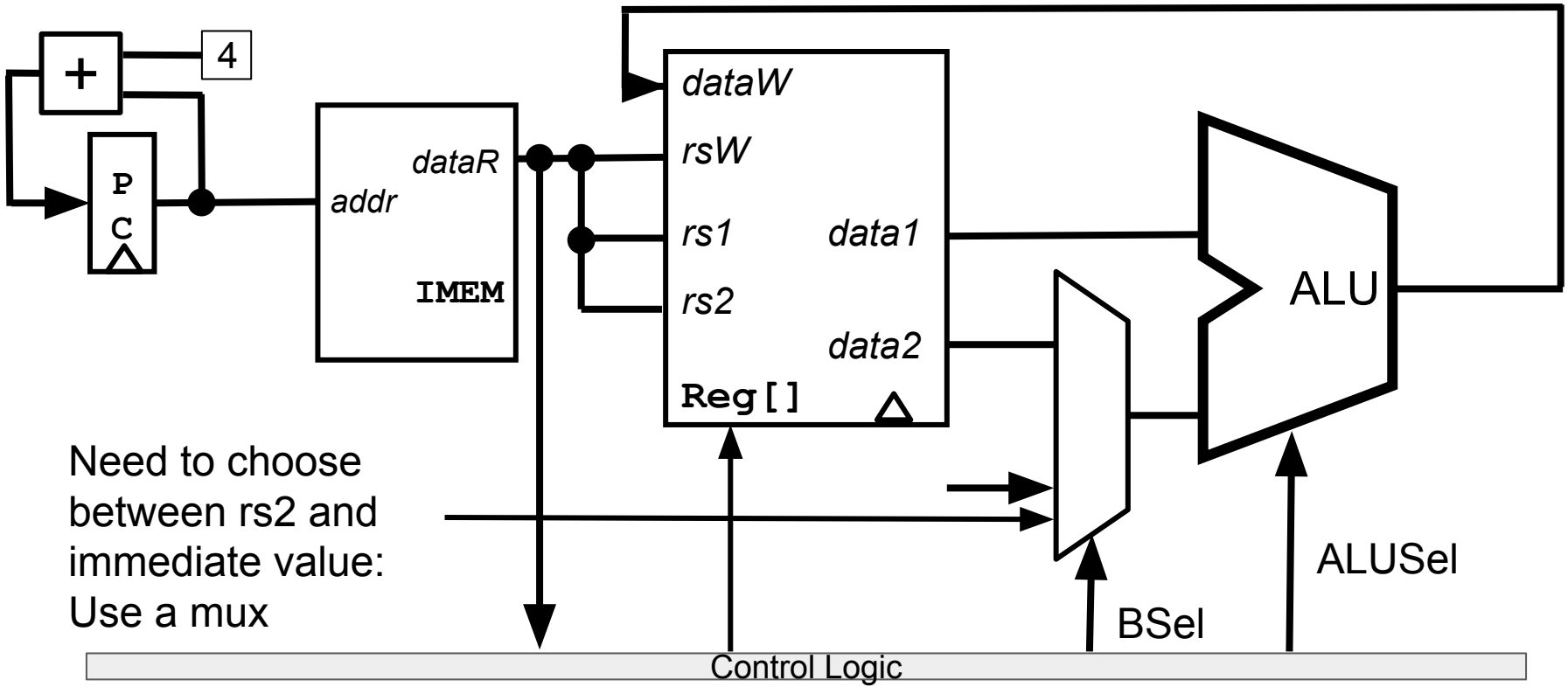
`addi rd rs1 imm`	ADD Immediate	`rd = rs1 + imm`	I	`001 0011`	`000`



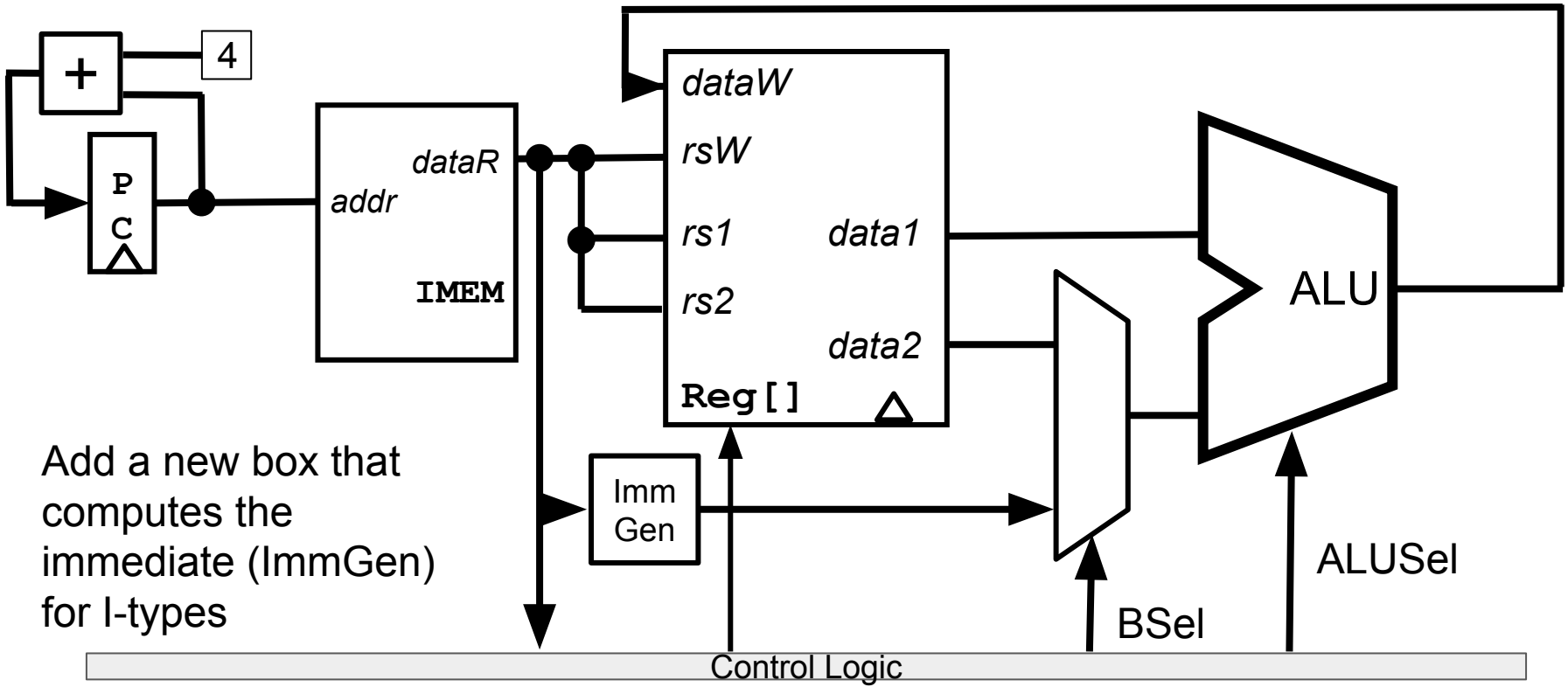
R-Type Datapath



Datapath with `addi`

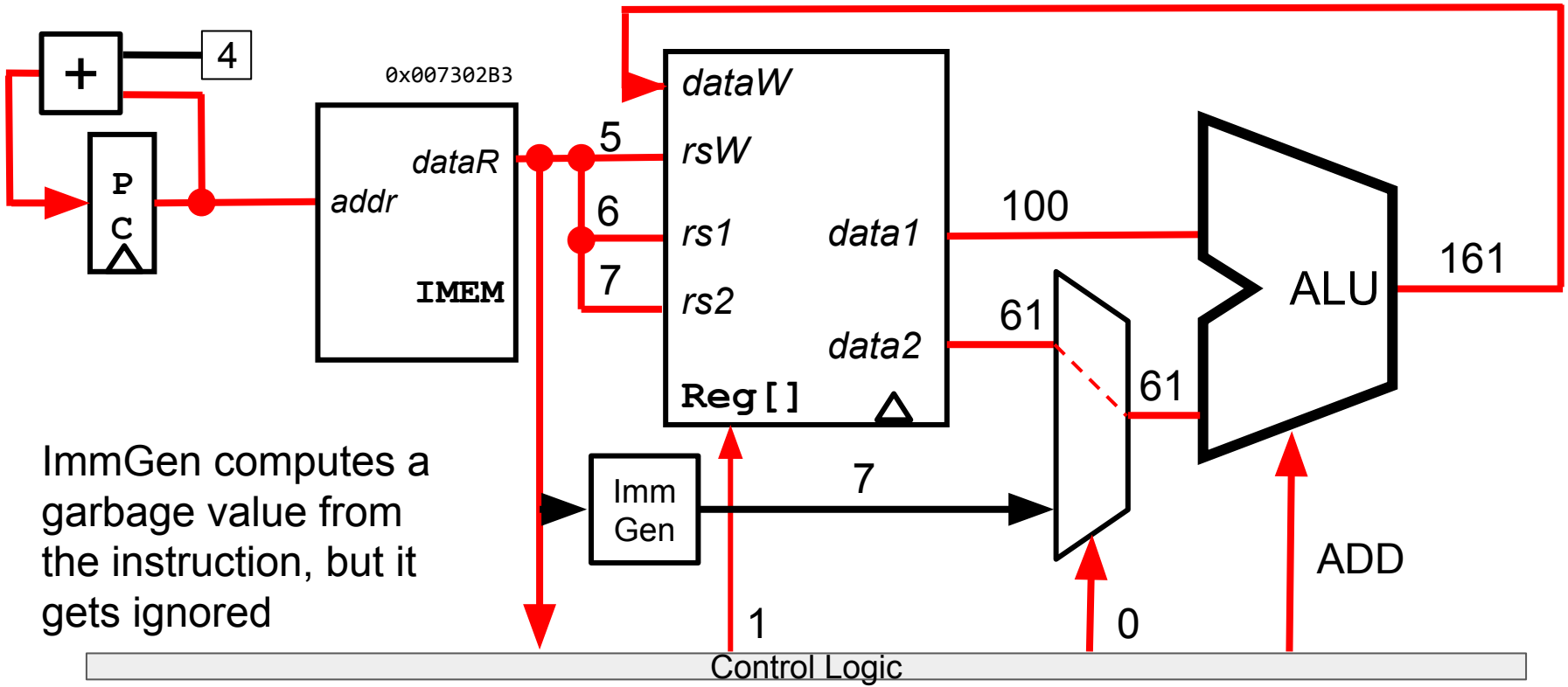


Datapath with `addi`



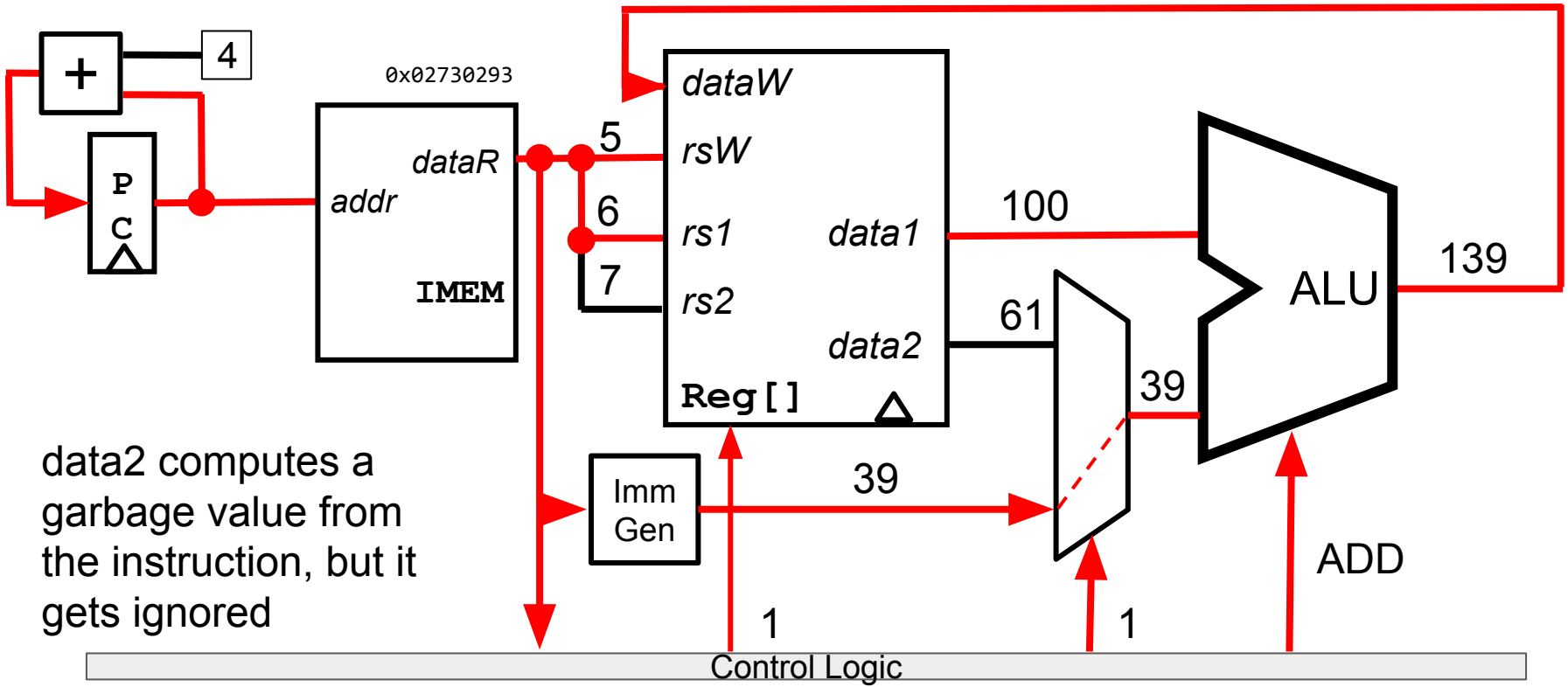
Add a new box that computes the immediate (ImmGen) for I-types

Datapath with `addi`: Running `add x5 x6 x7`



ImmGen computes a garbage value from the instruction, but it gets ignored

Datapath with `addi`: Running `addi x5 x6 39`

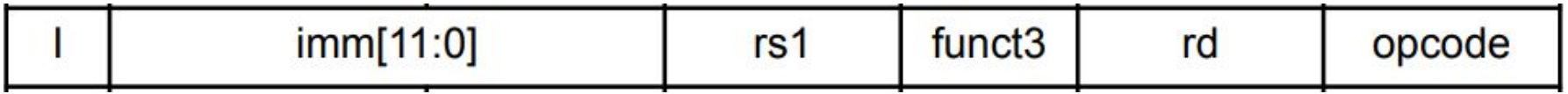


data2 computes a garbage value from the instruction, but it gets ignored

ImmGen design

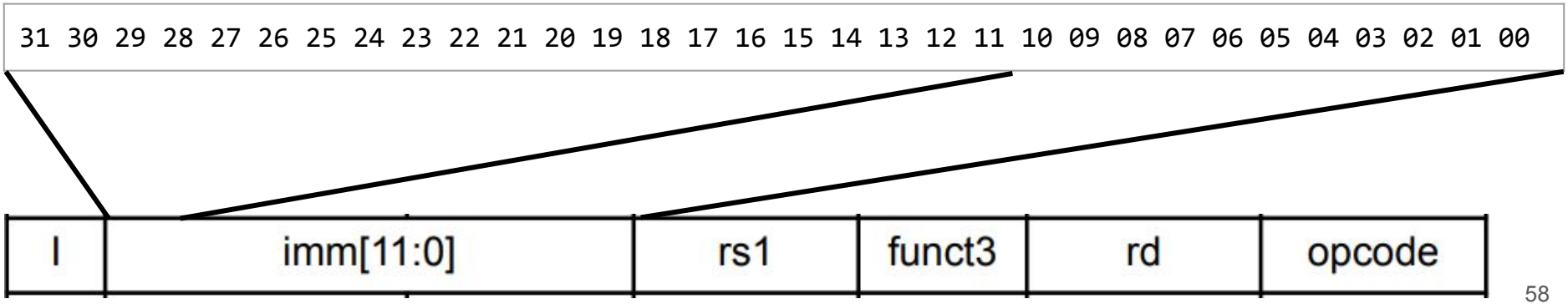
- Need to convert imm to a full 32-bit value (to send to ALU)

```
31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 09 08 07 06 05 04 03 02 01 00
```

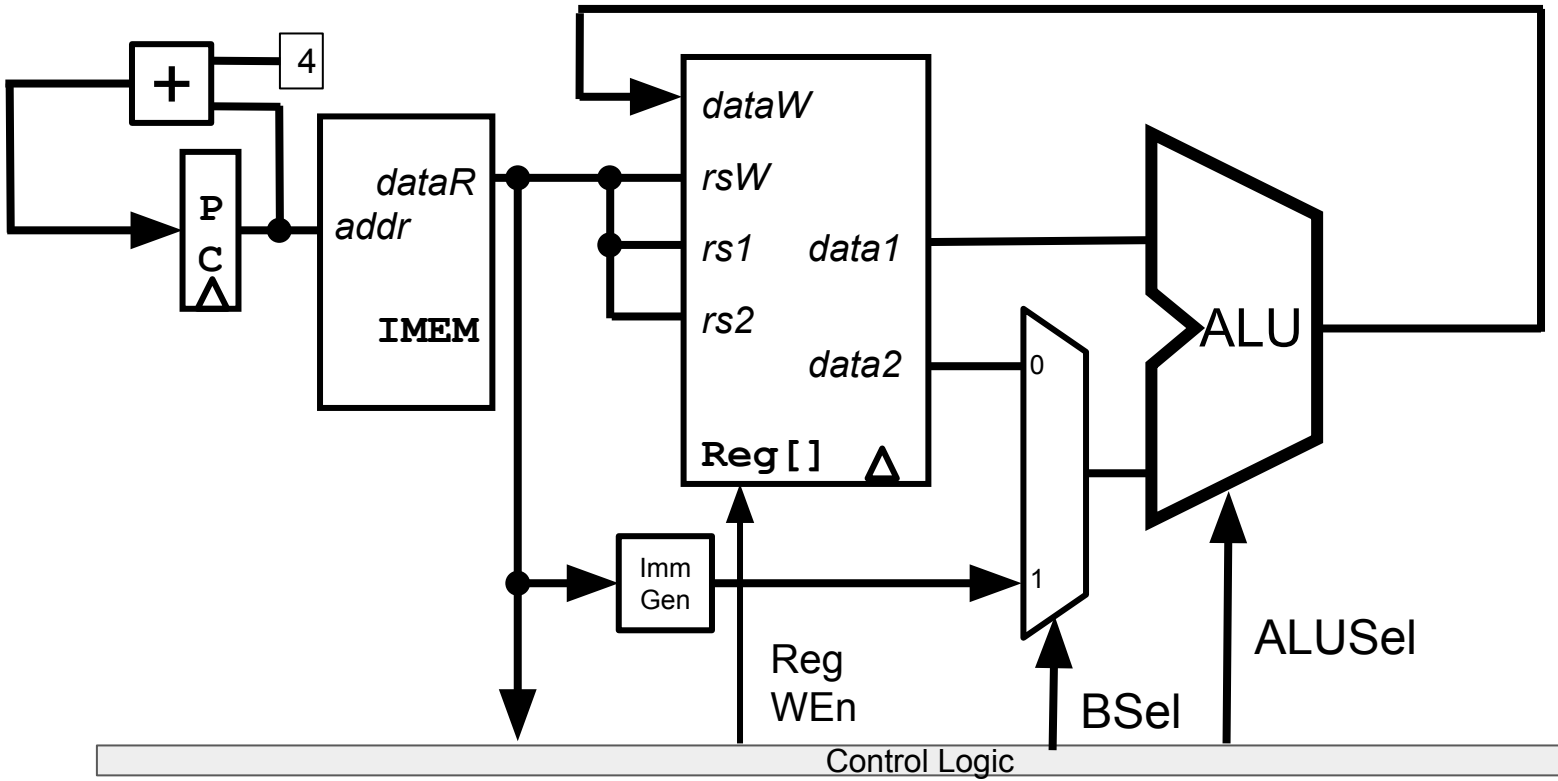


ImmGen design

- Need to convert imm to a full 32-bit value (to send to ALU)
- Copy the bottom 12 bits of imm to the bottom 12 bits of output
- Then *sign-extend* by copying bit 11 to all remaining bits of output
- In ALL instruction formats, bit 31 is the sign bit of the immediate (except R-types, which don't have an immediate)



Datapath so far



Agenda

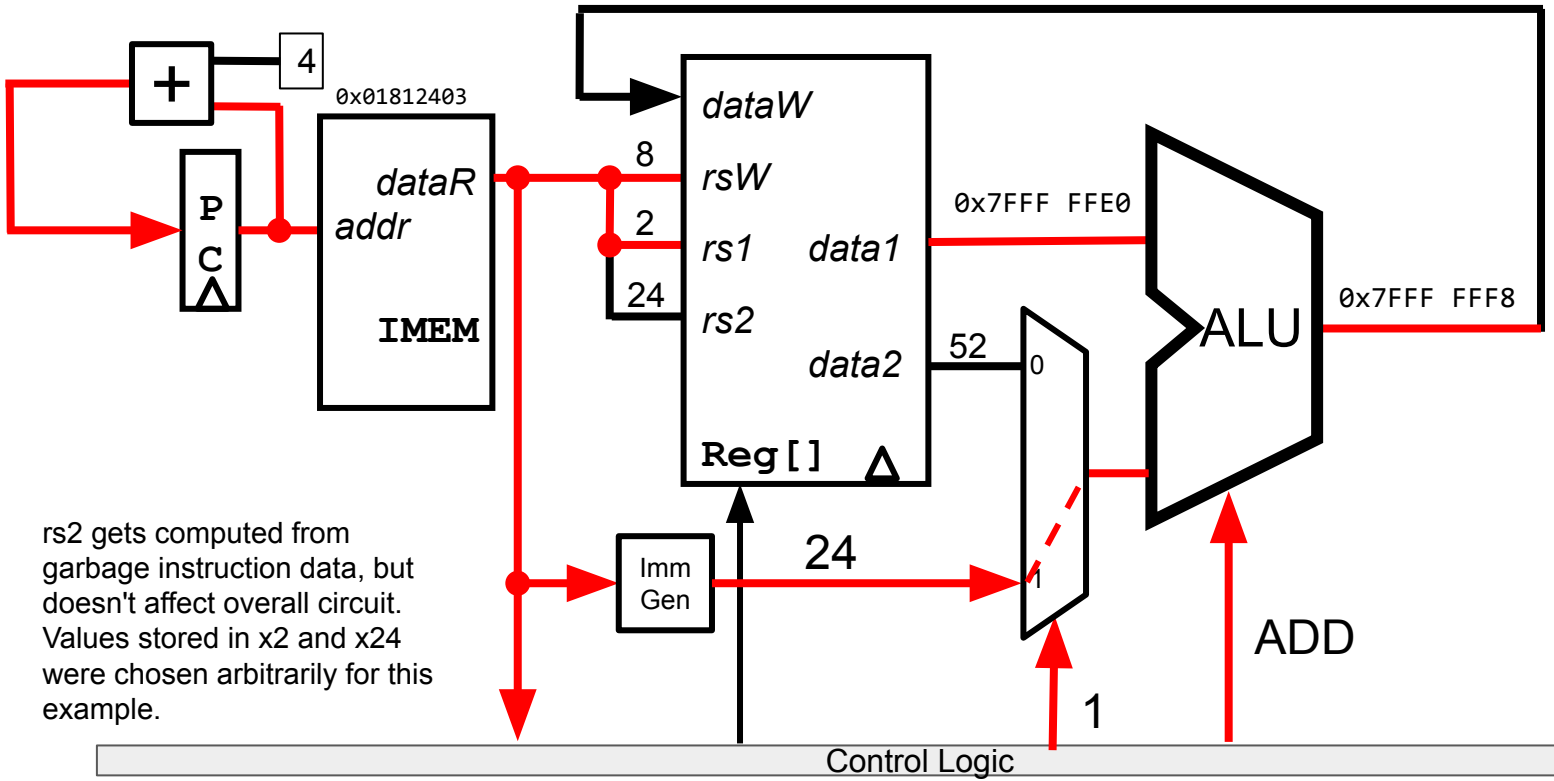
- Building a RISC-V Processor
- CPU Elements and Stages
- R-Type: `add` Datapath
- R-Type: `sub` Datapath
- Datapath with immediates: `addi`
- **Implementing Loads**
- Implementing Stores
- Implementing Jumps

Implementing the `lw` instruction

- Let's add `lw`
- `lw` computes one thing:
  - addr = rs1 + imm
    - Adding rs1+imm can be done using ALU in current datapath
- And affects two state elements:
  - rd = data at addr (4 bytes)
  - PC = PC+4
- Requires adding DMEM *after* ALU, but *before* writing back to register

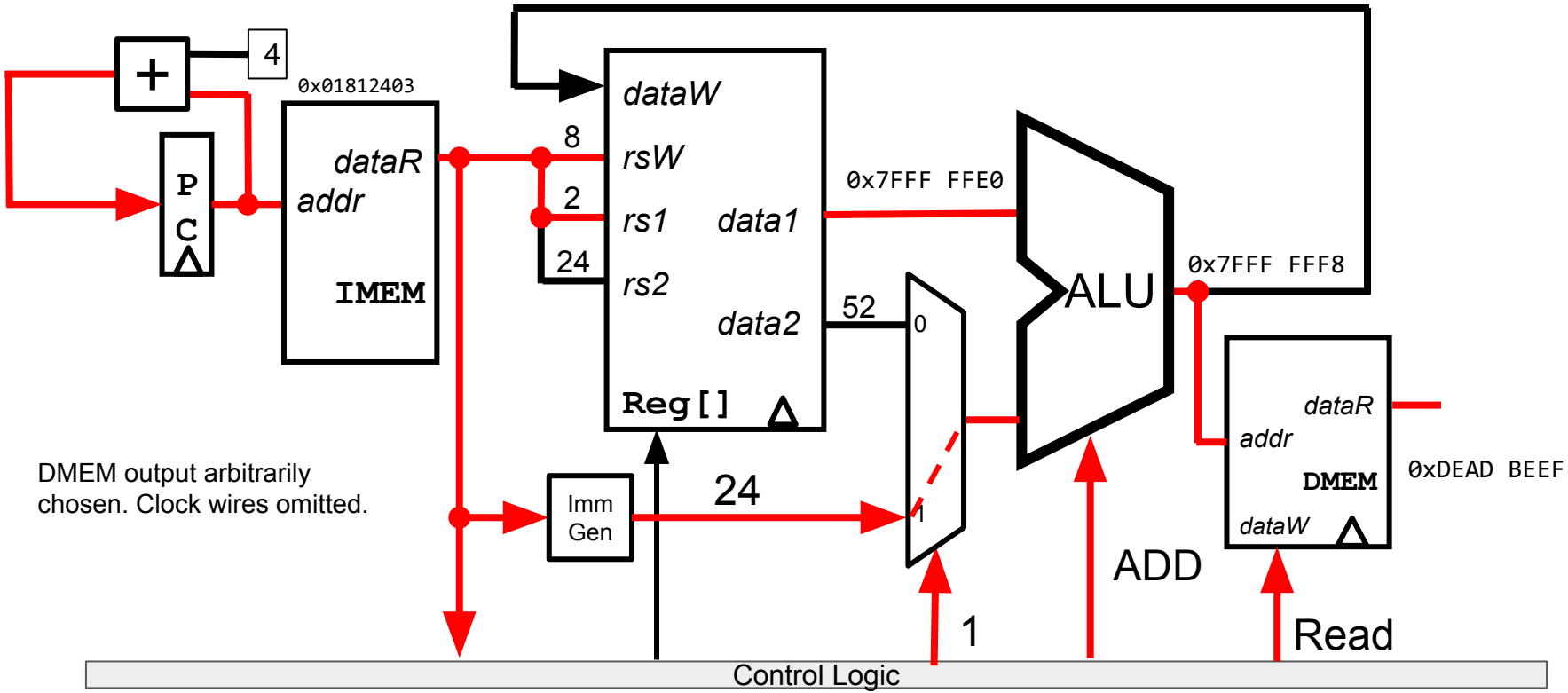
`lw rd imm(rs1)`	Load Word	`rd` = 4 bytes of memory starting at address `rs1 + imm`	I	`000 0011`	`010`	

Datapath running `lw x8 24(x2)`: IF, ID, EX



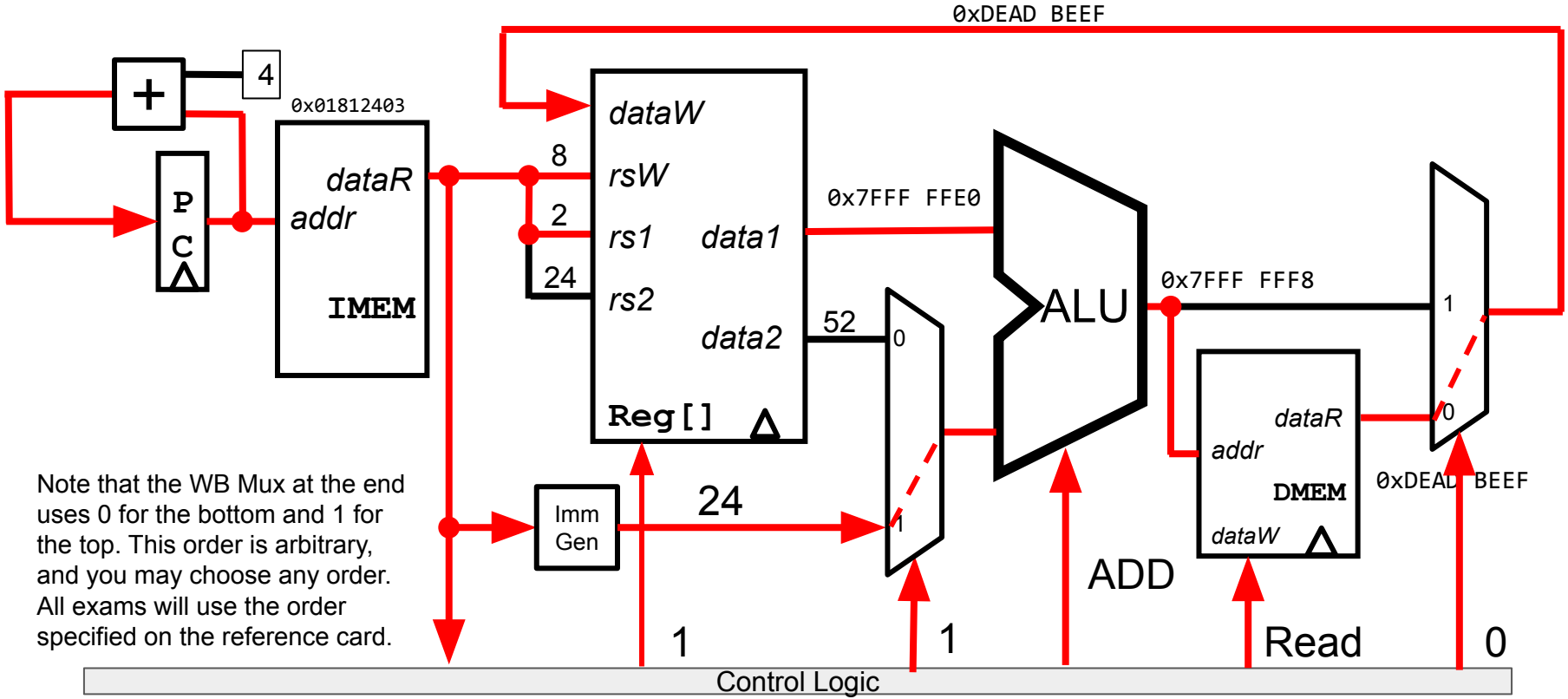
rs2 gets computed from garbage instruction data, but doesn't affect overall circuit. Values stored in x2 and x24 were chosen arbitrarily for this example.

Datapath running `lw x8 24(x2)`: IF, ID, EX, MEM



DMEM output arbitrarily chosen. Clock wires omitted.

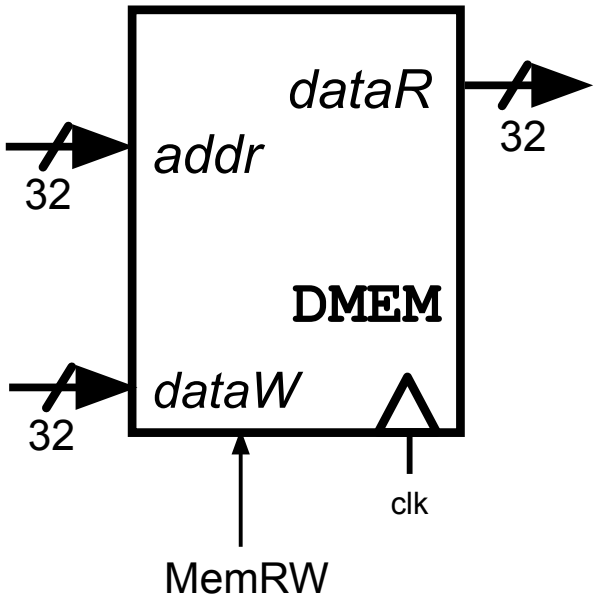
Datapath running `lw x8 24(x2)`



Note that the WB Mux at the end uses 0 for the bottom and 1 for the top. This order is arbitrary, and you may choose any order. All exams will use the order specified on the reference card.

DMEM Limitations

- To simplify its circuitry, DMEM can ONLY access 32 bits at a time
- Also many RISC-V DMEMs can only access addresses that **are a multiple of 4**.
  - Ex. If you want data at `0xFFFF FFFF`, you can access address `0xFFFF FFFC`, but not `0xFFFF FFFD`



DMEM Limitations Affect Programming

- Because of this inherent limitation, RISC-V often mandates that loads/stores happen on *aligned* addresses:
  - `lw/sw`: addr is multiple of 4
    - ex. `lw s0 3(sp)` is illegal unless `sp` is 1 mod 4
  - `lh/sh/lhu`: multiple of 2
  - `lb/sb/lbu`: can be run on any address
  - Other accesses yield undefined behavior: **the CPU can do anything if someone tries an unaligned access**

DMEM Limitations Affect Programming

- It's *possible* to create an unaligned access (pseudoinstruction-style) using only aligned accesses, but it's **much** slower.
  - Venus lets you use unaligned accesses despite this
  - Even in x86, which allows unaligned accesses, they take much longer to run
- This is why structs tend to place their components in a way that they're word-aligned, even if that wastes some space

DMEM Limitations Affect our Circuit: Loads

General procedure to handle loads:

1. Send into the DMEM the address of the word **containing your desired data**
    a. Since only aligned accesses are allowed, no load will ever require data from two different 4-byte blocks
    b. Ex. The byte starting at 0xDEADBEEF is contained with the 4-byte word starting at 0xDEADBEEC
2. Split the retrieved word into bytes/halfwords
3. Select the appropriate byte/halfword using address%4 and mux
4. Sign- or Zero-extend the resulting data to a full 32 bits

Exact implementation is an exercise in Project 3, and will be omitted for this lecture

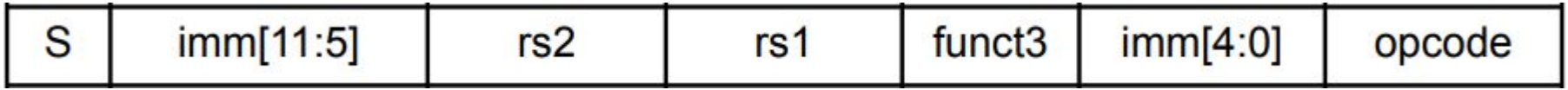
Agenda

- Building a RISC-V Processor
- CPU Elements and Stages
- R-Type: `add` Datapath
- R-Type: `sub` Datapath
- Datapath with immediates: `addi`
- Implementing Loads
- **Implementing Stores**
- Implementing Jumps

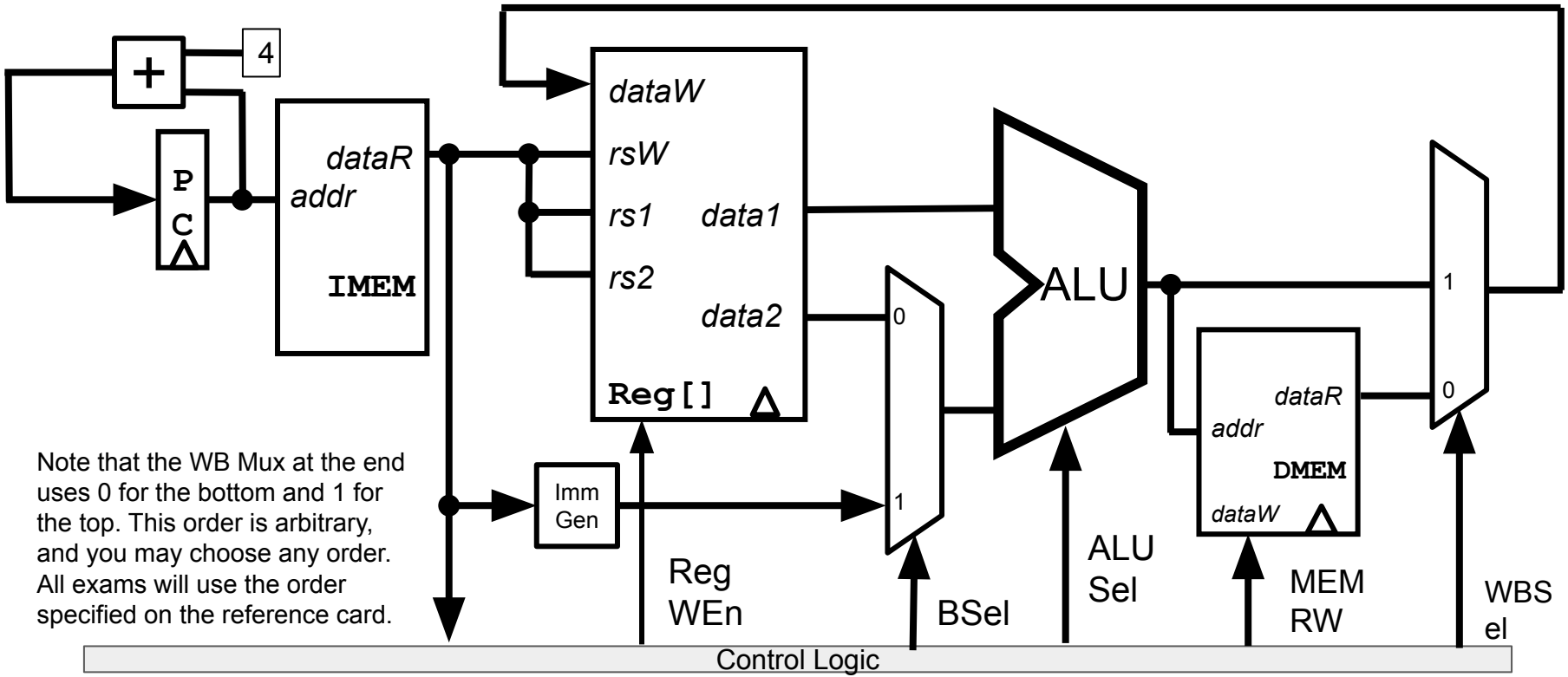
Implementing the `sw` instruction

- Let's add `sw`
- `sw` computes one thing:
  - addr = rs1 + imm
    - Note that this is the same as for `lw`. So we don't need to change our DMEM input.
- And affects two state elements:
  - DMEM[addr] = rs2
  - PC = PC+4
- Main issues:
  - imm is now split into two parts
  - DMEM write, no RegFile write

`sw rs2 imm(rs1)`	Store Word	Stores **`rs2`** starting at the address **`rs1 + imm`** in memory	S	`010 0011`	`010`	

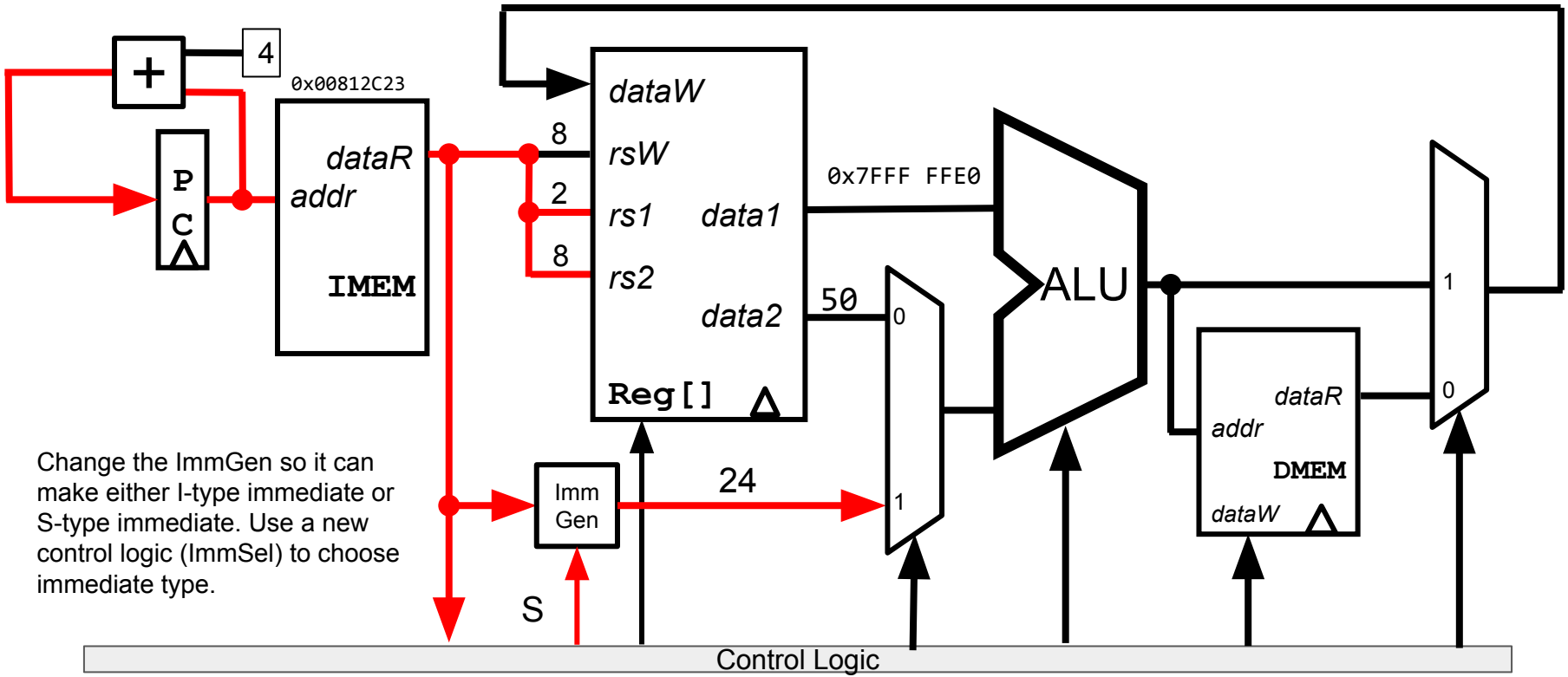


Datapath so far



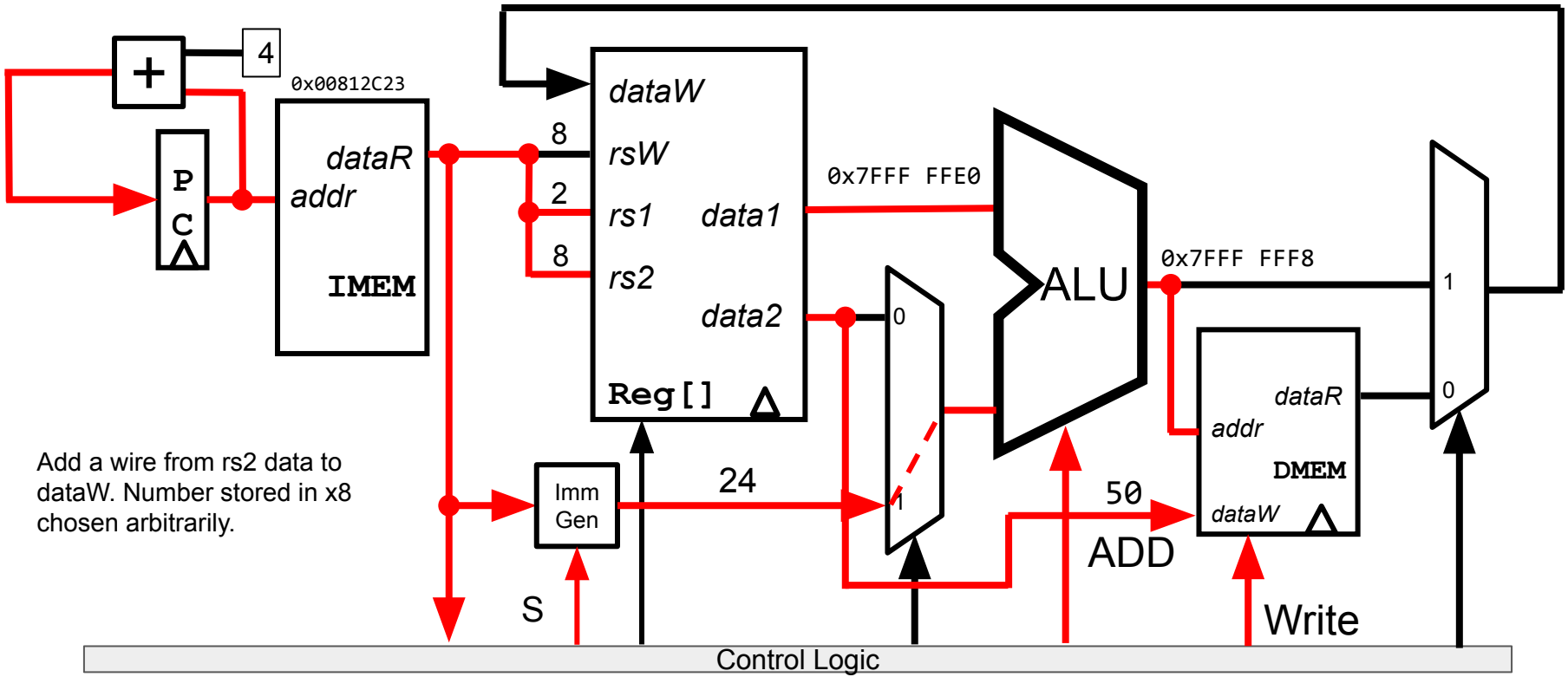
Note that the WB Mux at the end uses 0 for the bottom and 1 for the top. This order is arbitrary, and you may choose any order. All exams will use the order specified on the reference card.

Datapath running `sw s0 24(sp)`: Handling new imm type



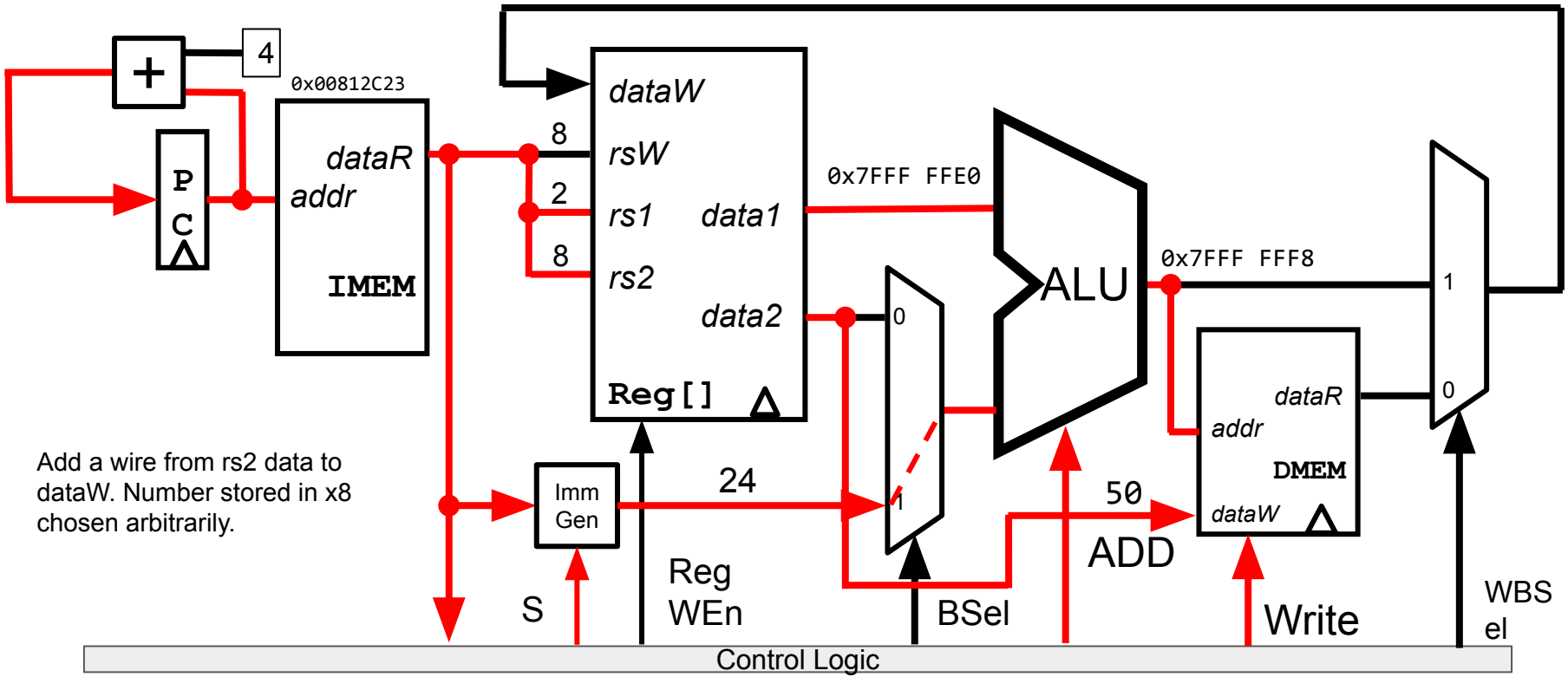
Change the ImmGen so it can make either I-type immediate or S-type immediate. Use a new control logic (ImmSel) to choose immediate type.

Datapath running `sw s0 24(sp)`: Sending data to DMEM



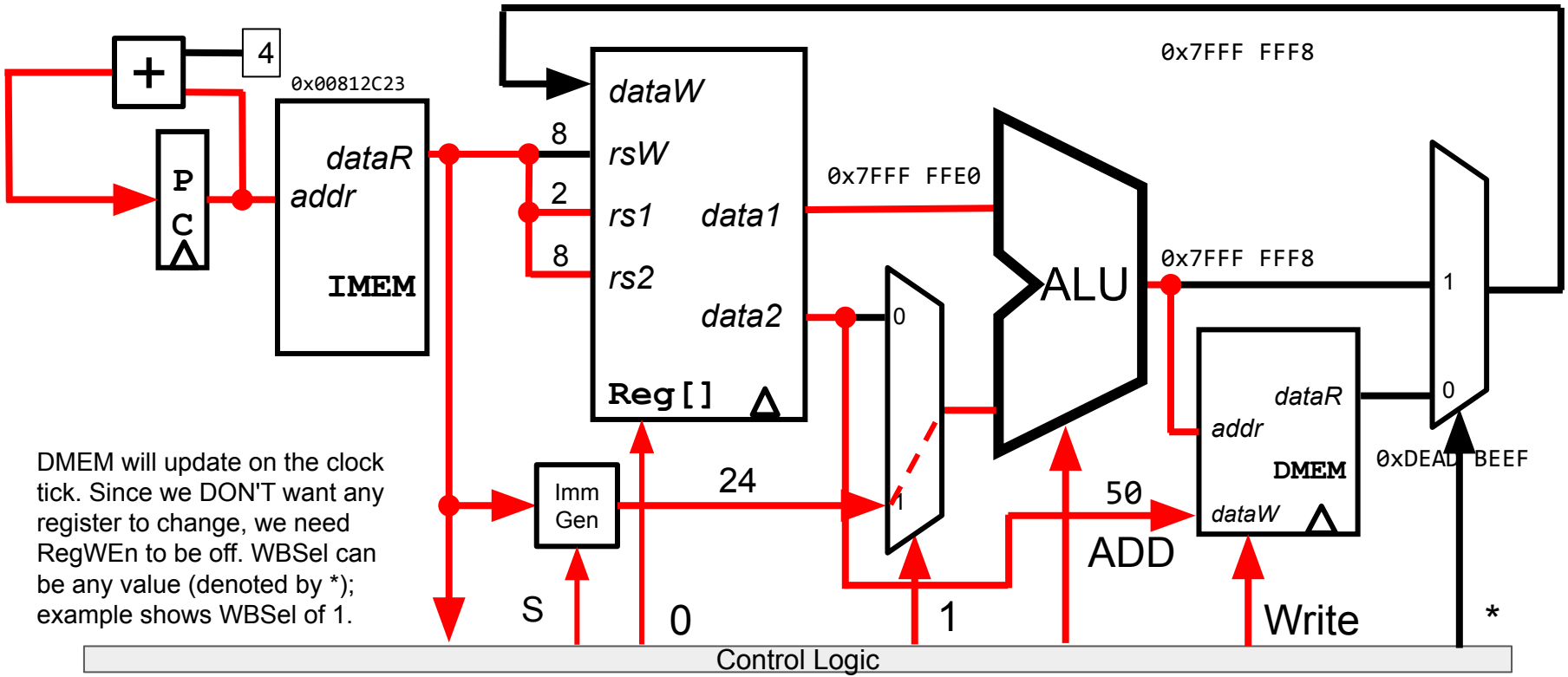
Add a wire from rs2 data to dataW. Number stored in x8 chosen arbitrarily.

What values do you set for RegWEn, BSel, WBSel?



Add a wire from rs2 data to dataW. Number stored in x8 chosen arbitrarily.

Datapath running `sw s0 24(sp)`: Write Back



DMEM will update on the clock tick. Since we DON'T want any register to change, we need RegWEn to be off. WBSel can be any value (denoted by *); example shows WBSel of 1.

DMEM Limitations Affect our Circuit: Stores

As with loads, DMEM's limitations affect our circuit design.

General approach:

- Determine which word contains the data we will store
  - As with loads, no store to two separate words
- Manipulate the data to align with the right bytes
  - Left-shifts according to the endianness
- Use separate read/write bits for each byte, so you don't overwrite unnecessary data

Exact implementation is an exercise in Project 3, and will be omitted for this lecture

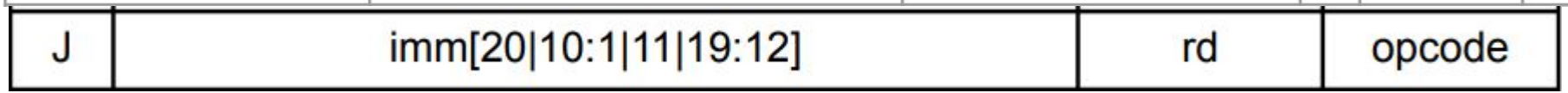
Agenda

- Building a RISC-V Processor
- CPU Elements and Stages
- R-Type: `add` Datapath
- R-Type: `sub` Datapath
- Datapath with immediates: `addi`
- Implementing Loads
- Implementing Stores
- **Implementing Jumps**

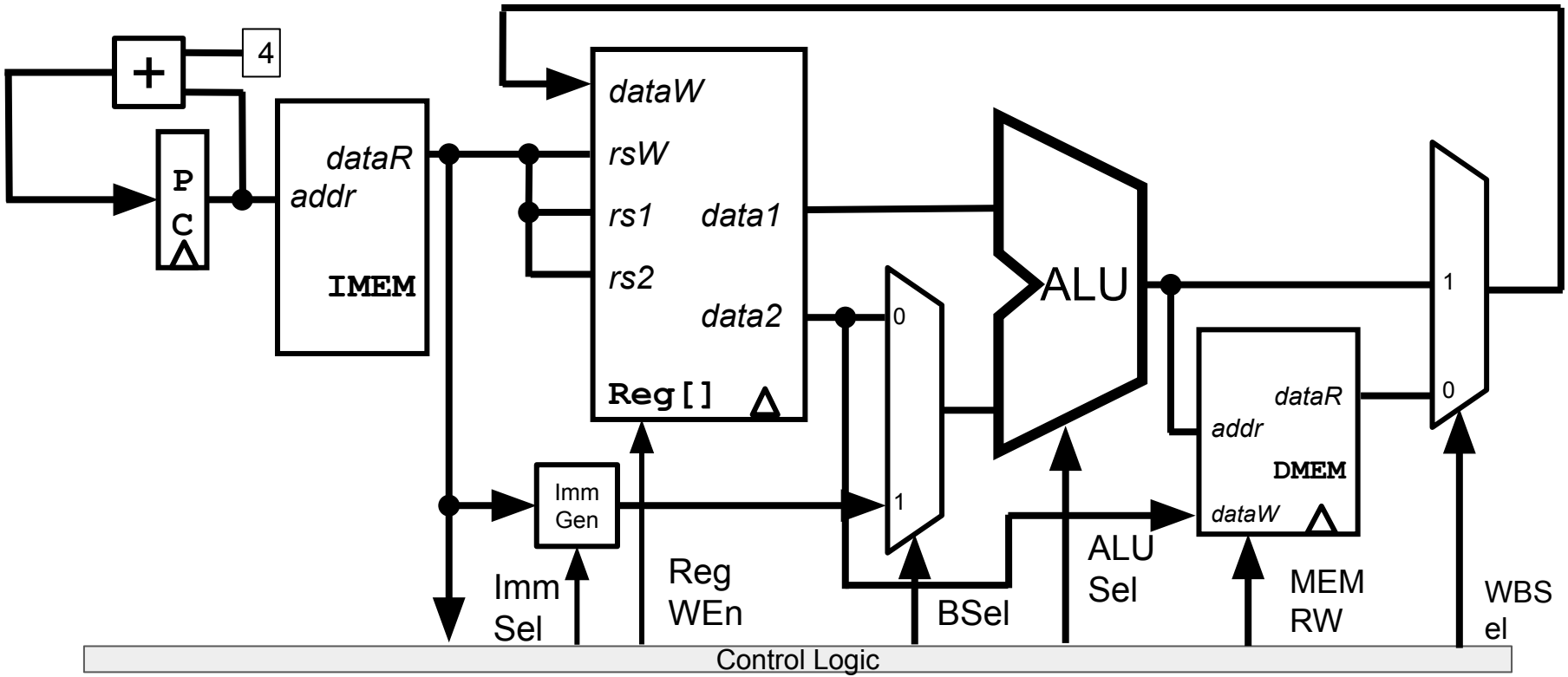
Implementing the `jal` instruction

- Let's add `jal`
- `jal` computes one thing:
  - New PC = PC + offset
    - Goal: Make the ALU do this math
- And affects two state elements:
  - rd =PC+4
  - PC = PC+offset
- Main issues:
  - ImmGen needs to handle J-type immediates (including adding the implicit 0)
  - WB to RegFile needs to include PC+4
  - WB to PC needs to include PC+offset
  - Send PC into ALU so it can compute PC+offset

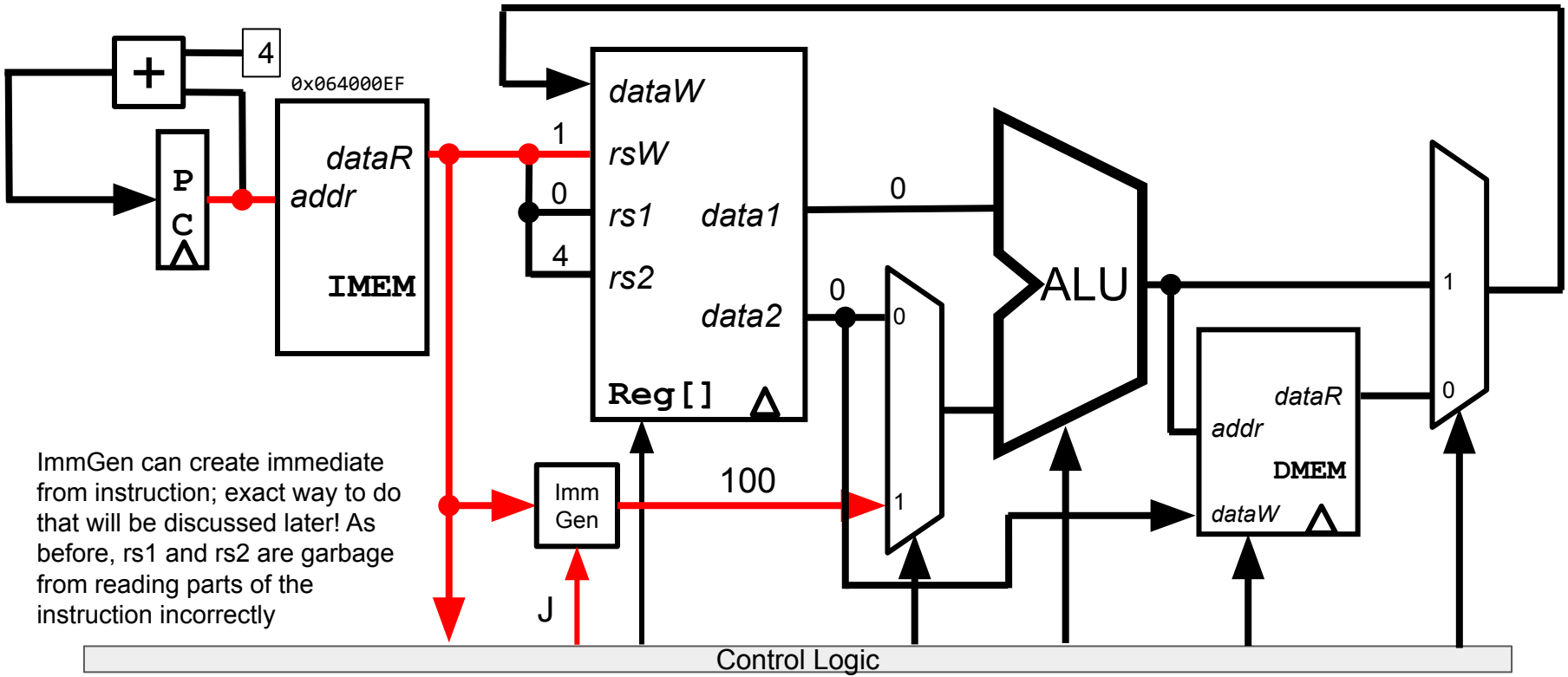
`jal rd label`	Jump And Link	`rd = PC + 4` `PC = PC + offset`	J	`110 1111`	



Datapath so far: how do you handle J-Types?

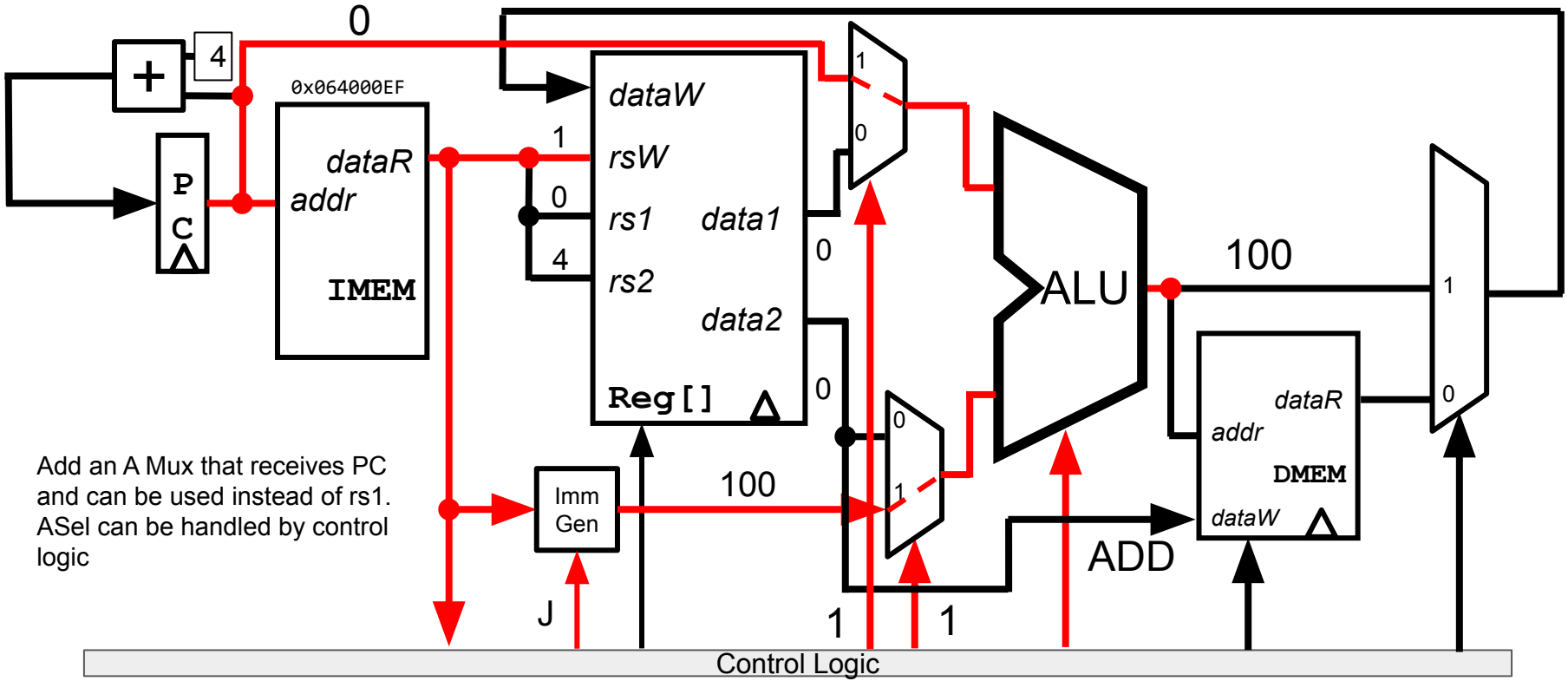


Datapath running `jal x1 100`: Add ImmSel for J



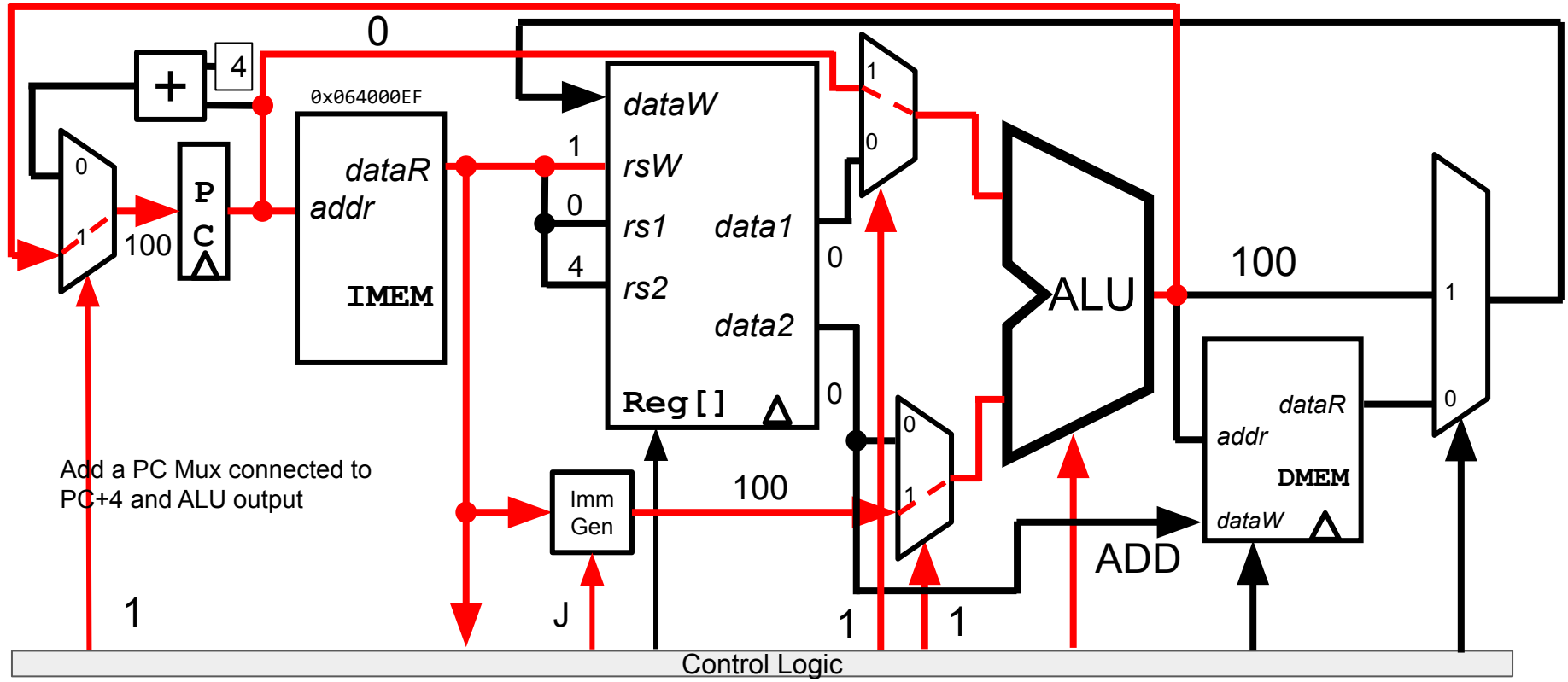
ImmGen can create immediate from instruction; exact way to do that will be discussed later! As before, rs1 and rs2 are garbage from reading parts of the instruction incorrectly

Datapath running `jal x1 100`: Computing PC+offset in ALU

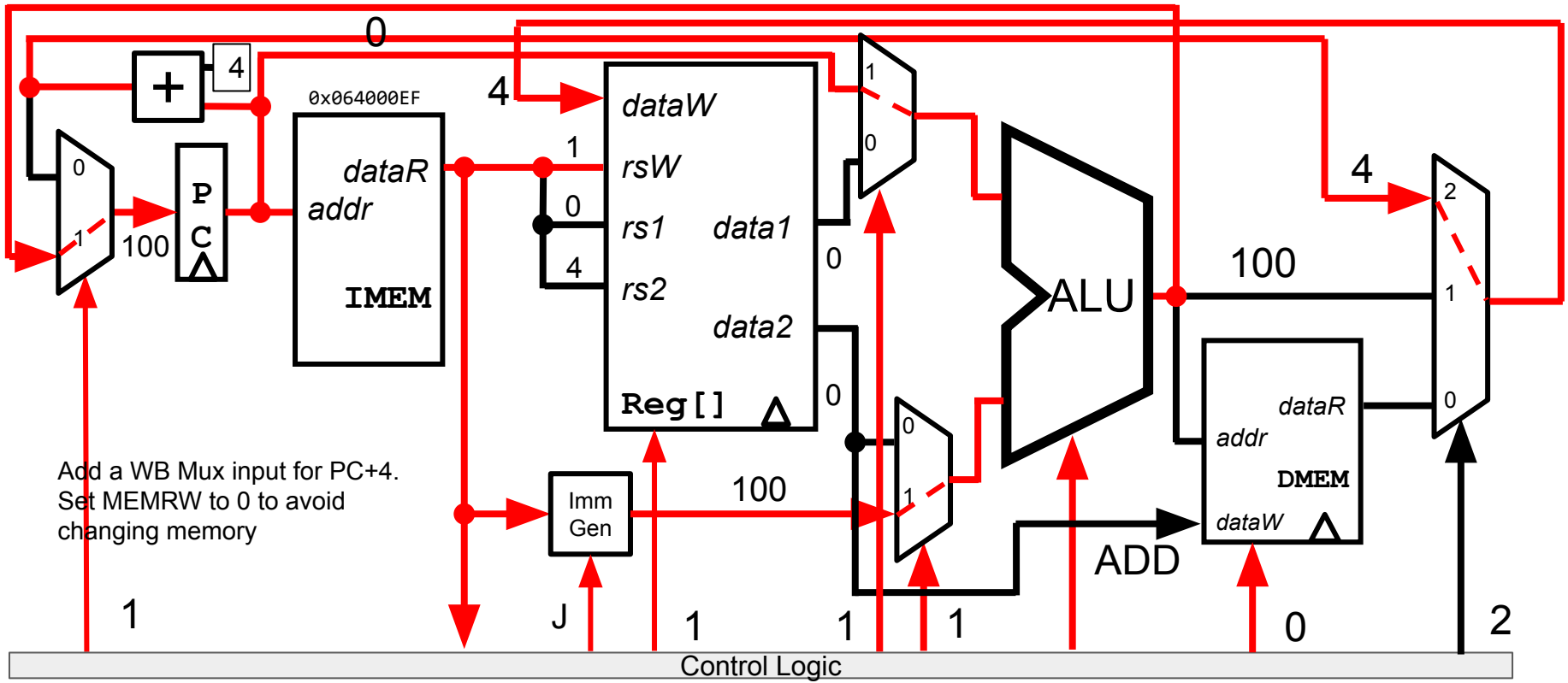


Add an A Mux that receives PC and can be used instead of rs1. ASel can be handled by control logic

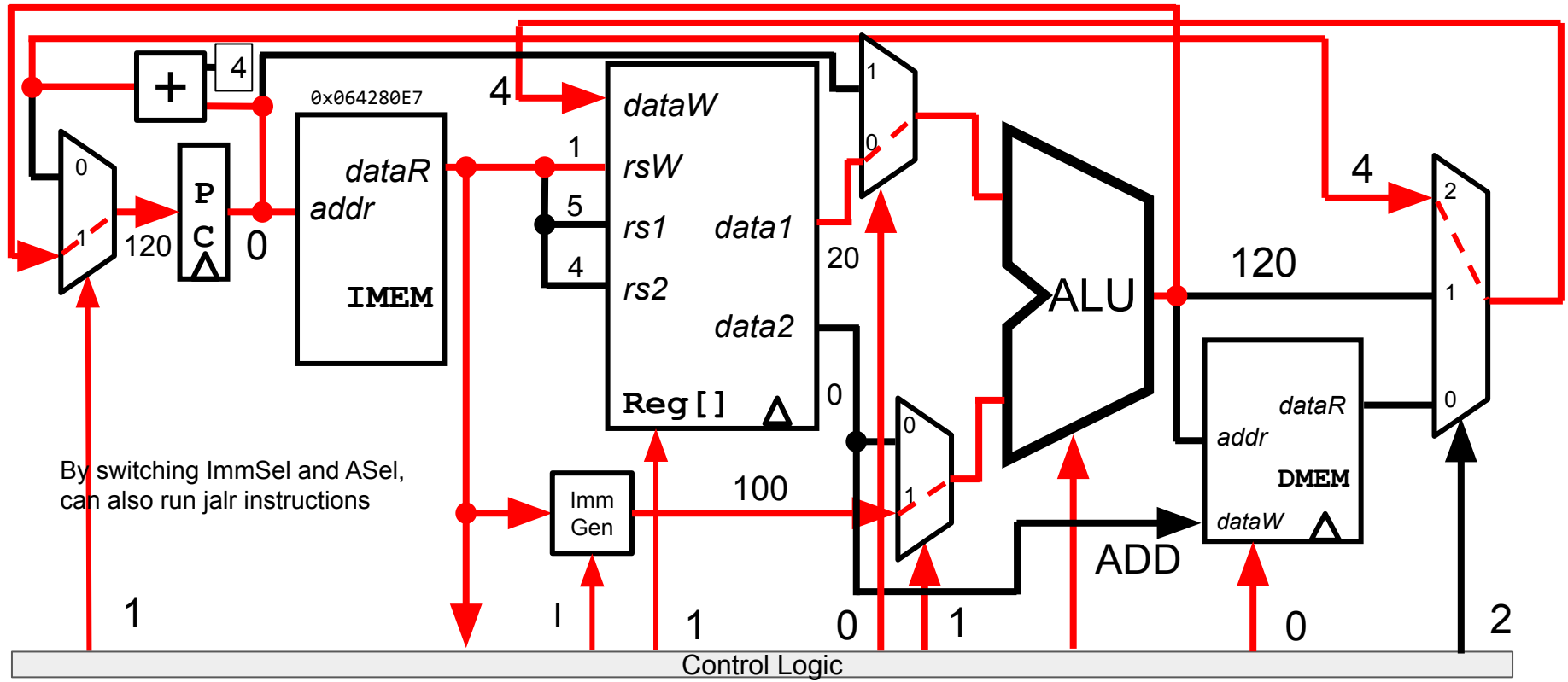
Datapath running `jal x1 100`: Setting PC to PC+Offset



Datapath running `jal x1 100`: Setting rd to PC+4



Datapath running `jalr x1 x5 100`



Datapath so far